OmniCode: Chapter 10: Track 1 - Verification & Functional Test Suites

Integrated Tripwires, Boot Tests & Functional Execution

CHAPTER IDENTIFIER
CH 10

NOTEBOOKLM WORD COUNT
11,312 words

CATALOGED CHAPTER FILES
6 files (2,673 lines)

GENERATION DATE
2026-10-16 20:36:02

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM GROUNDING FOCUS & LEARNING PROMPT

- **Chapter Focus:** *"How do tripwire tests prove zero-disk-mutation on dry run?"*
- **Architectural Rule:** All definitions conform to 4-block structure & spatial ruler.

CHAPTER MANIFEST & INDEX (6 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/assemble_tripwire_tests.rs**	RS	305	1,255	13.2
2	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/boot_tripwire_tests.rs**	RS	275	1,137	12.8
3	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/main.rs**	RS	433	1,808	19.6
4	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/platform_tripwire_tests.rs**	RS	241	1,214	11.6
5	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/functional/main.rs**	RS	593	2,494	26.5
6	**<TRACK 1> : {Host Engine & Virtual Machine}/tests/unit/main.rs**	RS	826	3,404	39.4

<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/assemble_tripwire_tests.rs RS | 305 LINES | 1255 WORDS | 13.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Assembler Tripwire, Adversarial Faults & Telemetry Oracle Manifest )
// ============================================================================
//! MODULE:              tests::integrated::assemble_tripwire_tests
//! IDENTITY:            Physical Assembler Adversarial Fault Injection & Telemetry Oracle Suite
//! NATURE:              Modular Integration Test Sandbox & Destructive Lowering Certifier
//! VERSION:             0.1.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {OmniCode Language & Standard Library} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Adversarial Assembler Oracle & Machine Lowering Tester
//!   WHAT:              Deliberately push the process tape assembler to fail under adversarial conditions:
//!                      missing files across 6-track roots, invalid file extensions, malformed grammar tapes,
//!                      unsealed commit invariants, and unwritable filesystem destinations.
//!   WHERE:             Track 1 Test Harness -> Assembler Lowering Engine -> DiagnosticLogger & Restorator
//!   WHEN:              Executed during comprehensive test suites and regression certification
//!   WHY:               Ensure that corrupted, malicious, or misconfigured process tapes never cause
//!                      unhandled panics or uncontrolled memory states, and that all failures emit
//!                      crisp balanced-ternary rollback telemetry (-1:ROLLBACK).
//!   HOW:               Adversarial fault injection ("push to fail"): if the test passes, it is
//!                      because the failure was caught and handled by tripwires, NOT because of soft checks.
//!
//! TOUCHES / FILESYSTEM FOOTPRINT:
//!   - LOGS:            Inscribed in-memory DiagnosticLogger ring buffers
//!   - DISK:            Temporary test tapes written to `/tmp` and cleaned up immediately
//!
//! TRIPWIRE REGISTRY:
//!   - `TRIPWIRE_INPUT_RESOLVE`       (Asserts missing file resolution trips rollback)
//!   - `TRIPWIRE_FILETYPE_VALIDATION` (Asserts non-tape/non-omni extensions trip rollback)
//!   - `TRIPWIRE_SYNTAX_TOKENIZATION` (Asserts malformed process ribbons trip rollback)
//!   - `TRIPWIRE_COMMIT_INVARIANT`    (Asserts unsealed commit status trips rollback)
//!   - `TRIPWIRE_OUTPUT_SERIALIZE`    (Asserts unwritable destination trips rollback)
//!   - `TRIPWIRE_DRY_RUN_GUARD`       (Asserts dry-run guarantees zero disk mutation)
//!   - `TRIPWIRE_BATON_PASS_WORDS`    (Asserts physical silicon word emission integrity)

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Test Fixtures & Environment Setup )
// ============================================================================
use std::fs;
use std::path::PathBuf;

use omnicode::cli::commands::assemble::{
    AssembleConfig, AssembleFormat, execute_assemble_pipeline, serialize_assembled_output,
};
use omnicode::core::filetype::resolve_track_path;
use omnicode::core::ternary::QuadState;
use omnicode::diagnostics::logger::{DiagnosticLogger, FailpointType};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Adversarial Tests: Pushing Assembler to Fail & Lowering Telemetry Oracles )
// ============================================================================

/// Test 1: Adversarial Process Tape Resolution Failure.
///
/// Deliberately queries a non-existent process tape across all 6 track roots.
/// Pushes `AssembleConfig::open` to fail, verifying that:
/// 1. A `TripwireError` is returned with channel `"TRIPWIRE_INPUT_RESOLVE"`.
/// 2. Failpoint classification is [`FailpointType::SyntaxAnomaly`].
/// 3. The recovery state is affirmative rollback (`-1:ROLLBACK`).
/// 4. The attached [`DiagnosticLogger`] in-memory trace registers the failpoint.
```

```rust
#[test]
fn test_adversarial_assemble_missing_file_trips_preflight() {
    let logger = DiagnosticLogger::new(false);
    let config = AssembleConfig::new("nonexistent_phantom_root/phantom_tape_999.tape");

    let open_result = config.open(&logger);
    assert!(open_result.is_err(), "Non-existent process tape MUST trip pre-flight resolution");

    let err = open_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_INPUT_RESOLVE");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    assert!(logger.has_failpoints(), "DiagnosticLogger must capture the failpoint");
    let fps = logger.failpoints();
    assert_eq!(fps.len(), 1);
    assert_eq!(fps[0].channel, "TRIPWIRE_INPUT_RESOLVE");
}

/// Test 2: Adversarial File Extension Rejection.
///
/// Provides a path that exists on disk (e.g. `Cargo.toml`), but has an unsupported
/// extension. Verifies that the assembler strictly refuses to assemble arbitrary files,
/// requiring `.tape` (process ribbon) or `.omni` (declarative spatial reel).
#[test]
fn test_adversarial_assemble_invalid_extension_trips() {
    let logger = DiagnosticLogger::new(false);
    let config = AssembleConfig::new("Cargo.toml");

    let open_result = config.open(&logger);
    assert!(open_result.is_err(), "Invalid file extension MUST trip pre-flight validation");

    let err = open_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_FILETYPE_VALIDATION");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    assert!(err.message.contains("unsupported extension"));
}

/// Test 3: Adversarial Malformed Process Ribbon Syntax.
///
/// Injects a syntactically broken tape containing unclosed boundaries and corrupted
/// directional vectors. Pushes `execute_assemble_pipeline` to fail, ensuring that
/// the lowering engine traps the error via [`FailpointType::SyntaxAnomaly`] without panicking.
#[test]
fn test_adversarial_assemble_corrupted_syntax_tape_trips() {
    let logger = DiagnosticLogger::new(false);
    let temp_tape_path = PathBuf::from("/tmp/omnicode_corrupted_syntax_test.tape");

    // Intentionally corrupted tape: unclosed delimiter and invalid token grammar
    let bad_source = r#"
[Tape.Corrupted]
    who: "AdversarialAgent",
    what: "InjectAnomaly"

TAPE HEAD:
    direction: +1:PROCEED,
    partition: 64

SUBORDINATE ROLLS:
    roll 1: [ BrokenBoundary < 42 ]
"#;

    fs::write(&temp_tape_path, bad_source).expect("Writing temporary corrupted tape");

    let config = AssembleConfig::new(&temp_tape_path);
    let mut ctx = config.open(&logger).expect("Preflight opening valid path");

    let assemble_result = execute_assemble_pipeline(&mut ctx);
    assert!(assemble_result.is_err(), "Corrupted tape syntax MUST trip assembly pipeline");

    let err = assemble_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_SYNTAX_TOKENIZATION");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    let _ = fs::remove_file(&temp_tape_path);
```

```
}

/// Test 4: Adversarial Commit Invariant Breach.
///
/// Injects a process tape whose execution or declaration does NOT conclude with
/// `commit_status [ PROCEED ]`. Asserts that `Tripwire::assert_quad_state_proceed`
/// fires on `"TRIPWIRE_COMMIT_INVARIANT"`, halting assembly.
#[test]
fn test_adversarial_assemble_unsealed_commit_status_trips() {
    let logger = DiagnosticLogger::new(false);
    let temp_tape_path = PathBuf::from("/tmp/omnicode_unsealed_commit_test.tape");

    // Tape explicitly concludes with Rollback instead of Proceed
    let rollback_tape = r#"
[Tape.RollbackTrap]
    who: "AdversarialTester",
    what: "TrapCommitInvariant"

TAPE HEAD:
    direction: -1:ROLLBACK,
    partition: 128

SUBORDINATE ROLLS:
    roll 1: execute [ NopOperation ]
    roll 2: commit_status [ ROLLBACK ]
"#;

    fs::write(&temp_tape_path, rollback_tape).expect("Writing temporary rollback tape");

    let config = AssembleConfig::new(&temp_tape_path);
    let mut ctx = config.open(&logger).expect("Preflight opening valid path");

    let assemble_result = execute_assemble_pipeline(&mut ctx);
    assert!(assemble_result.is_err(), "Unsealed commit status MUST trip invariant assertion");

    let err = assemble_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_COMMIT_INVARIANT");
    assert_eq!(err.failpoint_type, FailpointType::RollbackTriggered);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    let _ = fs::remove_file(&temp_tape_path);
}

/// Test 5: Adversarial Output Path Permission & Serialization Failure.
///
/// Points the output path to a directory that cannot be written (e.g. `/dev/null/forbidden/out.asm`).
/// Verifies that disk serialization catches the IO error, tripwires `"TRIPWIRE_OUTPUT_SERIALIZE"`,
/// and returns `-1:ROLLBACK` without panicking.
#[test]
fn test_adversarial_assemble_output_unwritable_path_trips() {
    let logger = DiagnosticLogger::new(false);
    let boot_tape = resolve_track_path("lib/kernel/boot.tape");
    if !boot_tape.exists() {
        return;
    }

    let config = AssembleConfig {
        input_path: boot_tape,
        output_path: Some(PathBuf::from("/dev/null/forbidden_nonexistent_dir/output.asm")),
        format: AssembleFormat::Listing,
        verbose: false,
        strict: false,
        dry_run: false,
    };

    let mut ctx = config.open(&logger).expect("Preflight opening valid boot tape");
    let assembled = execute_assemble_pipeline(&mut ctx).expect("Assembling boot tape");

    let serialize_result = serialize_assembled_output(&assembled, &config, &logger);
    assert!(serialize_result.is_err(), "Unwritable output destination MUST trip serialization");

    let err = serialize_result.unwrap_err();
    assert_eq!(err.channel, "TRIPWIRE_OUTPUT_SERIALIZE");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);
}

/// Test 6: Dry Run Safety Invariant (Zero Disk Mutation).
```

```rust
///
/// Verifies that when `--dry-run` is active, the assembler completes all lowering,
/// symbol resolution, and formatting, but GUARANTEES that the output file is NOT written.
#[test]
fn test_assemble_dry_run_guarantees_zero_disk_mutation() {
    let logger = DiagnosticLogger::new(false);
    let boot_tape = resolve_track_path("lib/kernel/boot.tape");
    if !boot_tape.exists() {
        return;
    }

    let ghost_output = PathBuf::from("/tmp/omnicode_ghost_dry_run_artifact.asm");
    if ghost_output.exists() {
        let _ = fs::remove_file(&ghost_output);
    }

    let config = AssembleConfig {
        input_path: boot_tape,
        output_path: Some(ghost_output.clone()),
        format: AssembleFormat::Words,
        verbose: false,
        strict: false,
        dry_run: true, // Dry run enabled
    };

    let mut ctx = config.open(&logger).expect("Preflight opening boot tape");
    let assembled = execute_assemble_pipeline(&mut ctx).expect("Lowering pipeline");

    let result = serialize_assembled_output(&assembled, &config, &logger);
    assert!(result.is_ok(), "Dry run serialization must succeed without error");
    assert!(!ghost_output.exists(), "Dry run MUST NOT mutate or create output file on disk");
}

/// Test 7: Baton Pass Integration to Physical Silicon Words.
///
/// Assembles `lib/kernel/boot.tape` and certifies that the resulting `AssembledTape`
/// contains non-trivial physical 64-bit machine words (`Word64`), verified registers,
/// and discrete machine opcodes, validating the complete baton pass to hardware execution.
#[test]
fn test_baton_pass_assemble_to_machine_words_verification() {
    let logger = DiagnosticLogger::new(false);
    let boot_tape = resolve_track_path("lib/kernel/boot.tape");
    if !boot_tape.exists() {
        return;
    }

    let config = AssembleConfig {
        input_path: boot_tape,
        output_path: None,
        format: AssembleFormat::Words,
        verbose: false,
        strict: false,
        dry_run: true,
    };

    let mut ctx = config.open(&logger).expect("Opening boot context");
    let assembled = execute_assemble_pipeline(&mut ctx).expect("Lowering pipeline");

    // Baton pass validation
    assert_eq!(assembled.commit_status, QuadState::Proceed);
    assert!(!assembled.instructions.is_empty(), "Instruction stream must not be empty");

    let words = assembled.to_machine_words();
    assert_eq!(words.len(), assembled.instructions.len());

    for (idx, word) in words.iter().enumerate() {
        // Assert that words are non-trivial physical containers
        assert_ne!(word.raw, 0xFFFFFFFF_FFFFFFFF, "Word {} must not be uninitialized sentinel", idx);
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Test Harness Verification Seal & Baton Pass )
// ============================================================================
// All 7 physical assembler adversarial tripwire tests sealed and certified.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/boot_tripwire_tests.rs

RS | 275 LINES | 1137 WORDS | 12.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Boot Inception Tripwire & Self-Healing Telemetry Test Suite Manifest )
// ============================================================================
//! MODULE:              tests::integrated::boot_tripwire_tests
//! IDENTITY:            Bare-Metal Boot Telemetry, Tripwire Invariants & Restorator Healing Suite
//! NATURE:              Modular Integration Test Sandbox & Telemetry Oracle
//! VERSION:             0.1.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1> ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Self-Healing Test Oracle
//!   WHAT:              Verify boot tripwire sensors, failpoint logging, debugger frame capture,
//!                      and automated restorative healing handoff to the Restorator.
//!   WHERE:             Track 1 Test Harness -> Boot Pipeline -> Diagnostics Ring Buffer
//!   WHEN:              Executed during test runs and regression test verification
//!   WHY:               Ensure that runtime anomalies during machine inception never panic,
//!                      produce rich flight-recorder telemetry, and feed the Restorator.
//!   HOW:               Simulate invariant breaches, inspect DiagnosticLogger and OmniDebugger,
//!                      and certify candidate healing via test oracles.
//!
//! TOUCHES / FILESYSTEM FOOTPRINT:
//!   - ARTIFACT:        Track 0 kernel build script and target image artifacts
//!   - LOGS:            Inscribed in-memory DiagnosticLogger ring buffers
//!
//! TRIPWIRE REGISTRY:
//!   - TRIPWIRE_TEST_PRECONDITION_PATH (Asserts path existence sensing)
//!   - TRIPWIRE_TEST_BINARY_RESOLUTION (Asserts hypervisor binary availability)
//!   - TRIPWIRE_TEST_REMEDIATION_HANDOFF (Asserts logger/debugger to Restorator pipeline)

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Test Harness Setup, Mock Fixtures & Scoped Imports )
// ============================================================================
use std::path::PathBuf;

use omnicode::cli::commands::boot::{
    BootArch, BootConfig, BootContext, synthesize_hypervisor_command,
    DEFAULT_BOOT_ARCH, DEFAULT_GRAPHICAL_MODE, REAL_MODE_BOOT_VECTOR,
    VGA_TEXT_FRAMEBUFFER_BASE, COM1_SERIAL_IO_BASE,
};
use omnicode::cogs::platform::HostPlatform;
use omnicode::cogs::tripwire::Tripwire;
use omnicode::core::filetype::resolve_track_path;
use omnicode::core::ternary::QuadState;
use omnicode::diagnostics::debugger::OmniDebugger;
use omnicode::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion};
use omnicode::diagnostics::restorator::Restorator;
use omnicode::parser::lexer::Lexer;
use omnicode::parser::parser::Parser;

/// Creates an isolated test context with a non-printing diagnostic logger
/// and an attached debugger configured with a retroaction depth threshold.
fn create_test_boot_context(arch: BootArch, graphical: bool) -> BootContext {
    let config = BootConfig::new(arch, graphical);
    let logger = DiagnosticLogger::new(false);
    config.open(logger).expect("Boot context must open under valid workspace")
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Tripwire Telemetry, Debugger Frame Capture & Restorator Self-Healing Tests )
// ============================================================================

```

```rust
/// Test 1: Asserts that an invalid or missing path immediately trips the tripwire,
/// inscribes a structured failpoint event into the [`DiagnosticLogger`], records microsecond
/// timestamps, and emits an episodic cognitive note tagged `#failpoint`.
#[test]
fn test_boot_tripwire_missing_path_records_telemetry() {
    let logger = DiagnosticLogger::new(false);
    let non_existent = PathBuf::from("/non_existent_silicon_kernel_image.bin");

    let result = Tripwire::assert_path_exists(
        &logger,
        "BOOT",
        &non_existent,
        "Simulated Missing Kernel Image",
    );

    assert!(result.is_err(), "Missing path must breach the tripwire boundary");
    let err = result.unwrap_err();
    assert_eq!(err.channel, "BOOT");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    // Verify logger captured the failpoint event
    assert!(logger.has_failpoints(), "Logger must register the trapped failpoint");
    let failpoints = logger.failpoints();
    assert_eq!(failpoints.len(), 1);
    assert!(failpoints[0].message.contains("Simulated Missing Kernel Image"));
    assert_eq!(failpoints[0].channel, "BOOT");

    // Verify cognitive notes for AI episodic memory
    let notes = logger.cognitive_notes();
    assert_eq!(notes.len(), 1);
    assert_eq!(notes[0].category, "INCIDENT");
    assert!(notes[0].tags.contains(&"failpoint".to_string()));
}

/// Test 2: Asserts that tripwiring with remediation telemetry equips the logger event
/// with a concrete [`FixSuggestion`] that the [`OmniDebugger`] and [`Restorator`] can consume.
#[test]
fn test_boot_tripwire_with_remediation_telemetry() {
    let logger = DiagnosticLogger::new(false);
    let mut debugger = OmniDebugger::new(logger.clone(), 10);

    let suggestion = FixSuggestion {
        kind: FixKind::AddBlockHeader,
        line: 1,
        col: 1,
        target_token: "".to_string(),
        suggested_patch: "// BLOCK 1: METADATA BLOCK\n".to_string(),
        rationale: "Inject missing 4-block metadata header before machine compilation".to_string(),
    };

    let err = Tripwire::trip_with_remediation(
        &logger,
        "BOOT",
        FailpointType::SyntaxAnomaly,
        "Kernel manifest header missing 4-block architecture scaffolding",
        suggestion.clone(),
    );

    assert_eq!(err.recovery_state, QuadState::Rollback);

    // Verify failpoint with remediation in logger
    let failpoints = logger.failpoints();
    assert_eq!(failpoints.len(), 1);
    let fp = &failpoints[0];
    assert!(fp.suggestion.is_some(), "Remediation suggestion must be attached");
    assert_eq!(fp.suggestion.as_ref().unwrap().kind, FixKind::AddBlockHeader);

    // Verify debugger registers the incident into its remediation queue
    debugger.remediation_queue.push(suggestion);
    assert_eq!(debugger.remediation_queue.len(), 1);
    assert_eq!(debugger.remediation_queue[0].kind, FixKind::AddBlockHeader);
}

/// Test 3: The Game Dev Loop Oracle — Simulates a boot-stage anomaly (broken source
/// script in boot loader), captures the tripwire event, hands the telemetry trace
/// off to the [`Restorator`], and verifies that the patch passes the test oracle.
#[test]
```

```rust
fn test_boot_anomaly_telemetry_to_restorator_self_healing_oracle() {
    let broken_boot_script = "boot_kernel < 1 ]";
    let logger = DiagnosticLogger::new(false);

    // 1. Lex and parse the broken boot script to trip the boundary
    let mut lexer = Lexer::new(broken_boot_script, logger.clone());
    let tokens = lexer.tokenize().expect("Lexing succeeds");
    let mut debugger = OmniDebugger::new(logger.clone(), 5);
    let mut parser = Parser::new(tokens, debugger.clone());
    let parse_res = parser.parse_program();
    assert!(parse_res.is_err(), "Delimiter mismatch must fail parsing");

    // 2. Verify logger trapped the boundary mismatch failpoint
    let failpoints = logger.failpoints();
    assert_eq!(failpoints.len(), 1);
    let incident = &failpoints[0];
    assert_eq!(incident.failpoint_type, Some(FailpointType::BoundaryMismatch));
    assert!(incident.suggestion.is_some());

    // 3. Define the Test Oracle simulating valid boot script verification
    let oracle = |candidate_code: &str| -> Result<(), String> {
        let test_logger = DiagnosticLogger::new(false);
        let mut test_lexer = Lexer::new(candidate_code, test_logger.clone());
        let test_tokens = test_lexer.tokenize().map_err(|e| format!("Lex error: {e}"))?;
        let test_debugger = OmniDebugger::new(test_logger.clone(), 5);
        let mut test_parser = Parser::new(test_tokens, test_debugger);
        test_parser.parse_program().map_err(|e| format!("Parse error: {e}"))?;
        if test_logger.has_failpoints() {
            return Err("Candidate caused failpoints".to_string());
        }
        Ok(())
    };

    // 4. Debugger executes restoration from the tripwire log event
    let healed_code = debugger
        .execute_restoration_from_log(broken_boot_script, incident, oracle)
        .expect("Self-healing restoration must be verified by oracle");

    assert_eq!(healed_code, "boot_kernel < 1 >\n");

    // 5. Restorator executes automated multi-pass verification with oracle
    let restorator = Restorator::new(logger.clone());
    let report = restorator.restore_and_verify_with_oracle(broken_boot_script, |code| {
        if code.contains("bad_boundary") {
            Err("Forbidden pattern".to_string())
        } else {
            Ok(())
        }
    });

    assert!(report.verified, "Restorator must verify patch with oracle");
    assert!(report.was_modified);

    // 6. Confirm episodic cognitive memory reflects verified repair
    let notes = logger.cognitive_notes();
    assert!(notes.iter().any(|n| n.category == "REPAIR_VERIFIED" || n.category == "ORACLE_CERTIFIED"));
}

/// Test 4: Verifies the hypervisor command synthesis matrix across all 3 architectures
/// (BIOS MBR, UEFI Long Mode, Multiboot ELF) in both headless and GUI modes.
#[test]
fn test_hypervisor_command_synthesis_matrix() {
    let _logger = DiagnosticLogger::new(false);

    // Case A: BIOS Headless
    let ctx_bios = create_test_boot_context(BootArch::Bios, false);
    let cmd_bios = synthesize_hypervisor_command(&ctx_bios).expect("Synthesize BIOS command");
    let args_bios: Vec<String> = cmd_bios.get_args().map(|s| s.to_string_lossy().into_owned()).collect();
    assert!(args_bios.contains(&"-drive".to_string()));
    assert!(args_bios.iter().any(|a| a.contains("format=raw,file=")));
    assert!(args_bios.contains(&"-display".to_string()));
    assert!(args_bios.contains(&"none".to_string()));
    assert!(args_bios.contains(&"-serial".to_string()));
    assert!(args_bios.contains(&"stdio".to_string()));

    // Case B: BIOS Graphical
    let ctx_bios_gui = create_test_boot_context(BootArch::Bios, true);
    let cmd_bios_gui = synthesize_hypervisor_command(&ctx_bios_gui).expect("Synthesize BIOS GUI");
```

```
    let args_bios_gui: Vec<String> = cmd_bios_gui.get_args().map(|s| s.to_string_lossy().into_owned()).collect();
    assert!(!args_bios_gui.contains(&"-display".to_string()));
    assert!(args_bios_gui.contains(&"-serial".to_string()));

    // Case C: UEFI Headless
    let ctx_uefi = create_test_boot_context(BootArch::Uefi, false);
    let cmd_uefi = synthesize_hypervisor_command(&ctx_uefi).expect("Synthesize UEFI command");
    let args_uefi: Vec<String> = cmd_uefi.get_args().map(|s| s.to_string_lossy().into_owned()).collect();
    assert!(args_uefi.contains(&"-bios".to_string()));
    assert!(args_uefi.iter().any(|a| a.contains("fat:rw:")));

    // Case D: Multiboot Headless
    let ctx_mb = create_test_boot_context(BootArch::Multiboot, false);
    let cmd_mb = synthesize_hypervisor_command(&ctx_mb).expect("Synthesize Multiboot command");
    let args_mb: Vec<String> = cmd_mb.get_args().map(|s| s.to_string_lossy().into_owned()).collect();
    assert!(args_mb.contains(&"-kernel".to_string()));
    assert!(args_mb.iter().any(|a| a.contains("omnicode_multiboot.elf")));
}

/// Test 5: Verifies that hardware constants align with physical silicon definitions:
/// - Real mode reset vector = `0x7C00`
/// - VGA text buffer base = `0xB8000`
/// - COM1 serial port I/O base = `0x3F8`
/// - Default boot architecture = BIOS
/// - Default graphical mode = false (headless)
#[test]
fn test_boot_hardware_constants_and_silicon_addresses() {
    assert_eq!(REAL_MODE_BOOT_VECTOR, 0x7C00);
    assert_eq!(VGA_TEXT_FRAMEBUFFER_BASE, 0xB8000);
    assert_eq!(COM1_SERIAL_IO_BASE, 0x3F8);
    assert_eq!(DEFAULT_BOOT_ARCH, BootArch::Bios);
    assert_eq!(DEFAULT_GRAPHICAL_MODE, false);

    let host = HostPlatform::current();
    assert!(!host.hypervisor_binary().is_empty());

    let script = resolve_track_path("kernel/build_all_kernels.sh");
    assert!(script.exists(), "Build script must resolve");
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Test Suite Sealing, Invariant Verification Proof & Postconditions )
// ============================================================================
// Boot inception tripwire integration test suite verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/main.rs RS 433 LINES 1808 WORDS 19.6 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Integration Test Suite Identity, Subsystem Interactions & Failpoint Invariants )
// ================================================================================
//! OmniCode Subsystem Integration Test Suite
//!
//! Verifies interactions between compiler components, runtime engines,
//! and the autonomous self-healing loop:
//!   - Boundary physics and asymmetric causality delimiters.
//!   - Failpoint traps (mismatches, leaks, immutable mutations, caveat vetoes).
//!   - Cyclic retroaction depth bounding (infinite rollback protection).
//!   - 4-Block structural linter validation.
//!   - Module import and core standard library resolution.
//!   - Connected Self-Healing Feedback Loop:
//!     Logger -> Structured Cognitive Trace -> Debugger/Restorator -> Test Oracle.

use omnicode::core::boundaries::BoundaryType;
use omnicode::core::ternary::QuadState;
use omnicode::diagnostics::logger::{DiagnosticLogger, FailpointType};
use omnicode::diagnostics::debugger::OmniDebugger;
use omnicode::diagnostics::linter::OmniLinter;
use omnicode::parser::lexer::Lexer;
use omnicode::parser::parser::Parser;
use omnicode::runtime::environment::{Environment, RuntimeValue};
use omnicode::runtime::evaluator::Evaluator;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Integration Test Harness Setup & Invariant Preconditions )
// ================================================================================
/// Constructs an integration test fixture containing a silent [`DiagnosticLogger`]
/// and an [`OmniDebugger`] with a custom retroaction limit threshold.
///
/// # Arguments
/// * `threshold` - Maximum retroactions permitted before triggering [`FailpointType::CyclicRetroaction`].
fn create_test_debugger(threshold: usize) -> (DiagnosticLogger, OmniDebugger) {
    let logger = DiagnosticLogger::new(false);
    let debugger = OmniDebugger::new(logger.clone(), threshold);
    (logger, debugger)
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Integration Cases: Failpoints, Imports, Self-Healing Loop )
// ================================================================================

/// Verifies asymmetric causality delimiters that govern directional influence:
/// - `{ ... ]`: Top-Down / Outside-In imposition by ambient context.
/// - `[ ... }`: Bottom-Up / Inside-Out emergence defined by internal denotation.
/// - Standard symmetric pairs: `[ ... ]` (Denotation), `< ... >` (Invariant), `{ ... }` (Connotation), `( ... )` (Aside).
#[test]
fn test_asymmetric_causality_boundaries() {
    assert_eq!(BoundaryType::from_delimiters('{', ']').unwrap(), BoundaryType::TopDown);
    assert_eq!(BoundaryType::from_delimiters('[', '}').unwrap(), BoundaryType::BottomUp);
    assert_eq!(BoundaryType::from_delimiters('[', ']').unwrap(), BoundaryType::Denotation);
    assert_eq!(BoundaryType::from_delimiters('<', '>').unwrap(), BoundaryType::Invariant);
    assert_eq!(BoundaryType::from_delimiters('{', '}').unwrap(), BoundaryType::Connotation);
    assert_eq!(BoundaryType::from_delimiters('(', ')').unwrap(), BoundaryType::Aside);
}

/// Verifies that closing a boundary with an illegal partner (e.g. `< ... ]`) triggers
/// a [`FailpointType::BoundaryMismatch`], records remediation telemetry, and creates
/// an episodic [`crate::diagnostics::logger::CognitiveNote`] for AI memory.
#[test]
fn test_failpoint_boundary_mismatch_illegal_crossing() {
    let source = "bad_boundary < 42 ]";
    let (logger, debugger) = create_test_debugger(5);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenization should succeed");

    let mut parser = Parser::new(tokens, debugger);
```

```
    let result = parser.parse_program();
    assert!(result.is_err(), "Parser must reject illegal boundary '< ... ]'");

    let failpoints = logger.failpoints();
    assert!(!failpoints.is_empty(), "Failpoint must be recorded in logger");
    assert_eq!(
        failpoints[0].failpoint_type,
        Some(FailpointType::BoundaryMismatch),
        "Expected FP_BOUNDARY_MISMATCH"
    );

    // Verify cognitive notes captured the incident for AI episodic memory
    let notes = logger.cognitive_notes();
    assert!(!notes.is_empty(), "Incident note must be recorded");
    assert_eq!(notes[0].category, "INCIDENT");
    assert!(notes[0].tags.contains(&"remediation_ready".to_string()));
}

/// Verifies that reaching EOF while delimiters remain open on the stack triggers
/// [`FailpointType::BoundaryLeak`] with a remediation patch proposing the missing closers.
#[test]
fn test_failpoint_boundary_leak_unclosed_delimiter() {
    let source = "unclosed_block [ 100 200";
    let (logger, debugger) = create_test_debugger(5);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenization should succeed");

    let mut parser = Parser::new(tokens, debugger);
    let result = parser.parse_program();
    assert!(result.is_err(), "Parser must reject unclosed boundary reaching EOF");

    let failpoints = logger.failpoints();
    assert!(
        failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::BoundaryLeak)),
        "Expected FP_BOUNDARY_LEAK"
    );
}

/// Verifies that attempting to rebind or mutate an entity declared inside `< ... >`
/// triggers [`FailpointType::ImmutableMutation`] and suggests converting it to `[ ... ]`.
#[test]
fn test_failpoint_immutable_mutation_attempt() {
    let source = r#"
        CONSTANT < 100 >
        CONSTANT [ 200 ]
    "#;
    let (logger, debugger) = create_test_debugger(5);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokens");

    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse");

    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let result = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog));

    assert!(result.is_err(), "Must reject mutation of invariant <...>");
    let failpoints = logger.failpoints();
    assert!(
        failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::ImmutableMutation)),
        "Expected FP_IMMUTABLE_MUTATION"
    );
}

/// Verifies that an unsatisfied adversative caveat expression (`! [ ... ]`) vetoes
/// the active state transition, triggers [`FailpointType::CaveatVeto`], and returns [`QuadState::Rollback`].
#[test]
fn test_failpoint_caveat_veto_and_rollback() {
    let source = "Transaction [ +1 ] ! [ -1 ]";
    let (logger, debugger) = create_test_debugger(5);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokens");

    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse");

    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let result = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
```

```
152
153     assert_eq!(result, RuntimeValue::State(QuadState::Rollback));
154
155     let failpoints = logger.failpoints();
156     assert!(
157         failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::CaveatVeto)),
158         "Expected FP_CAVEAT_VETO"
159     );
160 }
161
162 /// Verifies that evaluating into a void state without a resolving caveat triggers
163 /// [`FailpointType::NullTrap`] and halts execution into [`QuadState::Halt`].
164 #[test]
165 fn test_failpoint_null_trap_absorption() {
166     let source = "risky_operation [ +1 ] ! [ NULL ]";
167     let (logger, debugger) = create_test_debugger(5);
168     let mut lexer = Lexer::new(source, logger.clone());
169     let tokens = lexer.tokenize().expect("Tokens");
170
171     let mut parser = Parser::new(tokens, debugger);
172     let prog = parser.parse_program().expect("Parse");
173
174     let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
175     let result = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
176
177     assert_eq!(result, RuntimeValue::State(QuadState::Halt));
178
179     let failpoints = logger.failpoints();
180     assert!(
181         failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::NullTrap)),
182         "Expected FP_NULL_TRAP"
183     );
184 }
185
186 /// Verifies that repeated retroactions on the same label exceeding the threshold limit
187 /// are caught and aborted as [`FailpointType::CyclicRetroaction`].
188 #[test]
189 fn test_failpoint_cyclic_retroaction_threshold() {
190     let (logger, mut debugger) = create_test_debugger(3);
191
192     assert!(debugger.track_retroaction("loop").is_ok());
193     assert!(debugger.track_retroaction("loop").is_ok());
194     assert!(debugger.track_retroaction("loop").is_ok());
195
196     let over_threshold = debugger.track_retroaction("loop");
197     assert!(over_threshold.is_err(), "Must trip cyclic retroaction detector");
198
199     let failpoints = logger.failpoints();
200     assert!(
201         failpoints.iter().any(|fp| fp.failpoint_type == Some(FailpointType::CyclicRetroaction)),
202         "Expected FP_CYCLIC_RETROACTION"
203     );
204 }
205
206 /// Verifies structural linter enforcement of the universal 4-Block architecture:
207 /// Confirms that compliant code passes, and non-compliant code missing blocks is flagged with errors.
208 #[test]
209 fn test_four_block_structural_linter_validation() {
210     let logger = DiagnosticLogger::new(false);
211     let linter = OmniLinter::new(logger);
212
213     // Test compliant 4-block source
214     let compliant_source = r#"
215 # BLOCK 1: METADATA BLOCK
216 Module < Test > () {}
217 # BLOCK 2: OPENING BLOCK
218 Opening {}
219 # BLOCK 3: BODY BLOCK
220 Body []
221 # BLOCK 4: CLOSING BLOCK
222 Closing {}
223 "#;
224     let report = linter.lint_source("test_compliant", compliant_source);
225     assert!(report.is_valid, "Compliant 4-block source must pass linter");
226     assert_eq!(report.blocks_found.len(), 4);
227
228     // Test non-compliant source (missing body block)
229     let non_compliant_source = r#"
```

```
# BLOCK 1: METADATA BLOCK
Module < Broken > () {}
# BLOCK 2: OPENING BLOCK
Opening {}
# BLOCK 4: CLOSING BLOCK
Closing {}
"#;
    let fail_report = linter.lint_source("test_broken", non_compliant_source);
    assert!(!fail_report.is_valid, "Source missing Body block must fail lint");
    assert!(fail_report.errors.iter().any(|e| e.contains("Missing BLOCK 3")));
}

/// Verifies standard library imports (`Core.Arithmetic`, `Core.Logic`) inside the `Opening` block,
/// confirming that library macros (`ADD`, `SUB`, `PROCEED_MACRO`) are resolved and registered.
#[test]
fn test_module_import_and_library_resolution() {
    let source = r#"
        Opening {
            import: [ Core.Arithmetic :: Core.Logic ],
            preconditions: [ +1 ]
        }
        test_val [ 50 ]
        delta [ 10 ]
        res [ test_val --> test_val plus delta ]
        Closing {
            verify: [
                res = 60 ::
                ADD = InvariantMacro
            ],
            commit_status [ PROCEED ]
        }
    "#;
    let logger = DiagnosticLogger::new(false);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize module import");
    let debugger = OmniDebugger::new(logger, 10);
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse module import");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
    assert_eq!(res, RuntimeValue::State(QuadState::Proceed));

    assert!(evaluator.loaded_modules.contains("Core.Arithmetic"));
    assert!(evaluator.loaded_modules.contains("Core.Logic"));
    assert!(evaluator.env.contains("ADD"), "ADD macro must be present from Core.Arithmetic");
    assert!(evaluator.env.contains("SUB"), "SUB macro must be present from Core.Arithmetic");
    assert!(evaluator.env.contains("PROCEED_MACRO"), "PROCEED_MACRO must be present from Core.Logic");
    assert!(evaluator.debugger.immutable_registry.contains("ADD"));
    assert_eq!(evaluator.env.get("res"), Some(&RuntimeValue::Number(60.0)));
}

/// Verifies that the standard vector library `lib/core/vectors.omni` parses and executes cleanly.
#[test]
fn test_lib_core_vectors() {
    let path = omnicode::core::filetype::resolve_track_path("lib/core/vectors.omni");
    let source = std::fs::read_to_string(&path).unwrap();
    let logger = DiagnosticLogger::new(false);
    let mut lexer = Lexer::new(&source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize");
    let debugger = OmniDebugger::new(logger.clone(), 10);
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
    assert_eq!(res, RuntimeValue::State(QuadState::Proceed));
    assert!(!logger.has_failpoints(), "Failpoints: {:?}", logger.failpoints());
}

/// Verifies that the standard memory library `lib/core/memory.omni` parses and executes cleanly,
/// testing vessels, partitions, forward vector writes, and bounds verification.
#[test]
fn test_lib_core_memory() {
    let path = omnicode::core::filetype::resolve_track_path("lib/core/memory.omni");
    let source = std::fs::read_to_string(&path).unwrap();
    let logger = DiagnosticLogger::new(false);
    let mut lexer = Lexer::new(&source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize memory.omni");
    let debugger = OmniDebugger::new(logger.clone(), 10);
```

```
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse memory.omni");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval memory.omni");
    assert_eq!(res, RuntimeValue::State(QuadState::Proceed));
    assert!(!logger.has_failpoints(), "Failpoints: {:?}", logger.failpoints());
}

/// Verifies multi-pass automatic healing of syntax errors and unclosed delimiters
/// via [`crate::diagnostics::restorator::Restorator`].
#[test]
fn test_restorator_boundary_and_structural_auto_heal() {
    let broken_source = r#"
        bad_boundary < 42 ]
        unclosed_block [ 10 20
        step [ 5 --> 10 ]
    "#;
    let logger = DiagnosticLogger::new(false);
    let restorator = omnicode::Restorator::new(logger);
    let report = restorator.restore_source(broken_source);

    assert!(report.was_modified, "Broken source must be modified by restorator");
    assert!(!report.repairs_applied.is_empty(), "Restorator must log applied repairs");
    assert!(report.verified, "Restored code must pass verification: {:?}", report.errors);
    assert_eq!(report.final_state, Some(QuadState::Proceed));
    assert!(report.restored_content.contains("BLOCK 1: METADATA"));
    assert!(report.restored_content.contains("BLOCK 2: OPENING"));
    assert!(report.restored_content.contains("BLOCK 3: BODY"));
    assert!(report.restored_content.contains("BLOCK 4: CLOSING"));
}

/// Verifies end-to-end prompt transduction into executable AST and natural response generation
/// via [`crate::ai::SemanticTransducer`].
#[test]
fn test_ai_semantic_transducer_round_trip() {
    let prompt = "Add 1 ball to the group A which contains 4 balls";
    let logger = DiagnosticLogger::new(false);
    let transducer = omnicode::AITransducer::new(logger);
    let session = transducer.transduce_and_execute(prompt).expect("Transduction must succeed");

    assert_eq!(session.execution_state, QuadState::Proceed);
    assert!(session.synthesized_omnicode.contains("Ball < Entity? >"));
    assert!(session.synthesized_omnicode.contains("Group < Set? >"));
    assert!(session.synthesized_omnicode.contains("A [ 4 : Ball? ]"));
    assert!(session.synthesized_omnicode.contains("A --> A plus new_Ball"));
    assert!(session.synthesized_omnicode.contains("roll_1 = 5"));
    assert!(session.natural_response.contains("I added 1 ball to Group A"));
    assert!(session.natural_response.contains("5 balls in total"));
}

/// The Cybernetic Self-Healing Feedback Loop Test:
///
/// Demonstrates the full feedback loop between sensory logging, debugger intervention,
/// restorative patch synthesis, and test oracle verification:
/// 1. **Phase 1 (Anomaly Detection)**: Execution hits an illegal delimiter crossing (`< 99 ]`).
/// 2. **Phase 2 (Structured Telemetry)**: Logger records a [`FailpointType::BoundaryMismatch`]
///    equipped with a [`crate::diagnostics::logger::FixSuggestion`].
/// 3. **Phase 3 (Oracle-Verified Restoration)**: The Debugger consumes the structured log,
///    applies the patch, and runs an isolated test oracle to certify that the fix is sound.
/// 4. **Phase 4 (Immune System Restoration)**: Restorator executes round-trip verification with an oracle.
/// 5. **Phase 5 (Episodic Memory Audit)**: Verifies that cognitive memory notes reflect the certified repair.
#[test]
fn test_self_healing_feedback_loop_logger_debugger_oracle() {
    let broken_input = "rank1_invariant < 99 ]";
    let logger = DiagnosticLogger::new(false);

    // Phase 1: Anomaly Encounter & Structured Telemetry Inscription
    let mut lexer = Lexer::new(broken_input, logger.clone());
    let tokens = lexer.tokenize().expect("Lexing succeeds");
    let mut debugger = OmniDebugger::new(logger.clone(), 5);
    let mut parser = Parser::new(tokens, debugger.clone());
    let parse_err = parser.parse_program();
    assert!(parse_err.is_err(), "Must trap boundary mismatch");

    // Phase 2: Inspect Structured LogEvent and Remediation Suggestion
    let failpoints = logger.failpoints();
    assert_eq!(failpoints.len(), 1);
    let incident = &failpoints[0];
```

```
    assert_eq!(incident.failpoint_type, Some(FailpointType::BoundaryMismatch));
    assert!(incident.suggestion.is_some(), "Structured remediation required");

    // Phase 3: The Debugger applies the restoration from the structured log and invokes the test oracle
    let oracle = |candidate_code: &str| -> Result<(), String> {
        let test_logger = DiagnosticLogger::new(false);
        let mut test_lexer = Lexer::new(candidate_code, test_logger.clone());
        let test_tokens = test_lexer.tokenize().map_err(|e| format!("Lex error: {e}"))?;
        let test_debugger = OmniDebugger::new(test_logger.clone(), 5);
        let mut test_parser = Parser::new(test_tokens, test_debugger);
        test_parser.parse_program().map_err(|e| format!("Parse error: {e}"))?;
        if test_logger.has_failpoints() {
            return Err("Candidate caused failpoints".to_string());
        }
        Ok(())
    };

    let healed = debugger
        .execute_restoration_from_log(broken_input, incident, oracle)
        .expect("Self-healing restoration must be verified by oracle");

    assert_eq!(healed, "rank1_invariant < 99 >\n");

    // Phase 4: Restorator round-trip oracle certification
    let restorator = omnicode::Restorator::new(logger.clone());
    let report = restorator.restore_and_verify_with_oracle(broken_input, |code| {
        if code.contains("bad_boundary") {
            Err("Forbidden pattern".to_string())
        } else {
            Ok(())
        }
    });

    assert!(report.verified, "Restorator oracle certification succeeded");
    assert!(report.was_modified);

    // Phase 5: Confirm cognitive notes audit trace contains the verified repair
    let notes = logger.cognitive_notes();
    assert!(notes.iter().any(|n| n.category == "REPAIR_VERIFIED" || n.category == "ORACLE_CERTIFIED"));
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Integration Verification Proof & Postconditions )
// ============================================================================
// Integration test harness sealed and verified.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/integrated/platform_tripwire_tests.rs RS | 241 LINES | 1214 WORDS | 11.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Platform Adapter Tripwire, Adversarial Faults & Telemetry Oracle Manifest )
// ============================================================================
//! MODULE:              tests::integrated::platform_tripwire_tests
//! IDENTITY:            Platform Adapter Adversarial Fault Injection & Telemetry Oracle Suite
//! NATURE:              Modular Integration Test Sandbox & Destructive Telemetry Certifier
//! VERSION:             0.1.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {Engine Runtime & SVO} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Adversarial Platform Oracle & Telemetry Tester
//!   WHAT:              Deliberately push platform adapters to fail under hostile conditions,
//!                      inject faults, simulate missing hypervisors/firmware/display servers,
//!                      and certify that sensory tripwires produce structured telemetry and
//!                      remediation suggestions for the Restorator.
//!   WHERE:             Track 1 Test Harness -> Platform Adapter Cog -> DiagnosticLogger & Restorator
//!   WHEN:              Executed during comprehensive test suites and regression certification
//!   WHY:               Ensure that hostile host environments never cause unhandled panics or crashes,
//!                      and that every failure is converted into an actionable cognitive observation.
//!   HOW:               Adversarial fault injection ("push to fail"): if the test passes, it is
//!                      because the failure was caught and handled, NOT because of soft green checks.
//!
//! TOUCHES / FILESYSTEM FOOTPRINT:
//!   - LOGS:            Inscribed in-memory DiagnosticLogger ring buffers
//!   - SUBPROCESS:      Executes isolated test child processes to verify error trapping
//!
//! TRIPWIRE REGISTRY:
//!   - TRIPWIRE_TEST_HYPERVISOR_MISSING    (Asserts hypervisor missing remediation)
//!   - TRIPWIRE_TEST_FIRMWARE_UNRESOLVED   (Asserts firmware missing AI observation)
//!   - TRIPWIRE_TEST_DISPLAY_HEADLESS      (Asserts display server missing caveat fix)
//!   - TRIPWIRE_TEST_SUBPROCESS_NONZERO    (Asserts child process non-zero exit trapping)
//!   - TRIPWIRE_TEST_BATON_PASS_BOOT       (Asserts clean handoff to boot context)

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Test Fixtures & Environment Setup )
// ============================================================================
use std::process::Command;

use omnicode::cli::commands::boot::{BootArch, BootConfig};
use omnicode::cogs::platform::HostPlatform;
use omnicode::cogs::tripwire::Tripwire;
use omnicode::core::ternary::QuadState;
use omnicode::diagnostics::logger::{DiagnosticLogger, FailpointType};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Adversarial Tests: Pushing Platform Adapters to Fail & Telemetry Oracles )
// ============================================================================

/// Test 1: Adversarial Hypervisor Resolution Failure.
///
/// Deliberately simulates a non-existent or corrupted hypervisor binary on the host platform.
/// Pushes `Tripwire::assert_binary_available` to fail, verifying that:
/// 1. A `TripwireError` is returned with `QuadState::Rollback`.
/// 2. The `DiagnosticLogger` captures the failpoint with channel `"PLATFORM"`.
/// 3. A concrete `FixSuggestion` with installation instructions (`apt`, `brew`, `winget`)
///    is attached for the [`omnicode::diagnostics::restorator::Restorator`].
#[test]
fn test_adversarial_hypervisor_missing_trips_remediation() {
    let logger = DiagnosticLogger::new(false);
    let platform = HostPlatform::current();

```

```
    // Query an intentionally non-existent executable name
    let non_existent_binary = "qemu-system-nonexistent-silicon-arch-999";
    let check = platform.resolve_in_path(non_existent_binary);
    assert!(check.is_none(), "Non-existent binary must NOT resolve on PATH");

    // Push tripwire to fail and assert that it trips
    let trip_result = Tripwire::assert_binary_available(&logger, "PLATFORM", non_existent_binary);
    assert!(trip_result.is_err(), "Missing binary MUST trip the sensory wire");

    let err = trip_result.unwrap_err();
    assert_eq!(err.channel, "PLATFORM");
    assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
    assert_eq!(err.recovery_state, QuadState::Rollback);

    // Assert that the logger recorded the failpoint
    assert!(logger.has_failpoints(), "Logger must register the trapped failpoint");
    let fps = logger.failpoints();
    assert_eq!(fps.len(), 1);
    assert!(fps[0].message.contains(non_existent_binary));

    // Assert that cognitive note with #failpoint tag was recorded
    let notes = logger.cognitive_notes();
    assert!(
        notes.iter().any(|n| n.tags.iter().any(|t| t == "failpoint")),
        "Episodic trace must contain #failpoint tag"
    );
}

/// Test 2: Adversarial Firmware Traversal & Fallback Guidance.
///
/// Traverses firmware candidates when none are present on disk (or in a simulated environment),
/// asserting that `find_ovmf_firmware_with_telemetry`:
/// 1. Emits individual telemetry probe events into the logger.
/// 2. Records an AI cognitive observation labeled `"FIRMWARE_UNRESOLVED"`.
/// 3. Inscribes tags `["firmware", "ovmf", "uefi", "fallback"]` so the Restorator
///    and AI reasoning agents can propose falling back to `--bios` MBR mode.
#[test]
fn test_adversarial_firmware_discovery_records_ai_observation() {
    let logger = DiagnosticLogger::new(false);
    let platform = HostPlatform::Unknown; // Unknown platform has generic fallback candidates

    // Probing firmware with telemetry must record probe logs
    let fw = platform.find_ovmf_firmware_with_telemetry(&logger);

    let events = logger.events();
    assert!(events.iter().any(|e| e.channel == "FIRMWARE"), "Probe events must be logged");

    // If firmware is absent on host, verify AI observation was registered
    if fw.is_none() {
        let notes = logger.cognitive_notes();
        assert!(
            notes.iter().any(|n| n.category == "FIRMWARE_UNRESOLVED"),
            "Logger must record FIRMWARE_UNRESOLVED note for AI cortex and Restorator"
        );
        let target_note = notes.iter().find(|n| n.category == "FIRMWARE_UNRESOLVED").unwrap();
        assert!(target_note.tags.iter().any(|t| t == "fallback"));
    }
}

/// Test 3: Adversarial Display Server Assertion.
///
/// Tests `assert_display_server_available` when display server is absent (simulated or real).
/// Asserts that if no display server is active:
/// 1. The tripwire fires with `FailpointType::SyntaxAnomaly`.
/// 2. A `FixSuggestion` with `FixKind::AdjustCaveat` is generated, recommending `--headless`.
#[test]
fn test_adversarial_display_server_assertion_contract() {
    let logger = DiagnosticLogger::new(false);
    let platform = HostPlatform::current();

    // If display server is active, assert success
    if platform.has_display_server() {
        let res = platform.assert_display_server_available(&logger);
        assert!(res.is_ok(), "Display server assertion must succeed when display server is active");
    } else {
        // If headless, assert that tripwire trips with remediation
        let res = platform.assert_display_server_available(&logger);
        assert!(res.is_err(), "Headless environment MUST trip display server assertion");
```

```
        let err = res.unwrap_err();
        assert_eq!(err.channel, "DISPLAY");
        assert!(logger.has_failpoints(), "Logger must capture display failpoint");
    }
}

/// Test 4: Adversarial Host Subprocess Non-Zero Exit Trapping.
///
/// Deliberately spawns a child process that exits with a non-zero exit status code (`42`).
/// Asserts that `execute_host_command`:
/// 1. Traps the non-zero exit status without panicking.
/// 2. Returns a `TripwireError` containing the exit code and stderr output.
/// 3. Inscribes the failpoint into the logger with channel `"SUBPROCESS_TEST"`.
#[test]
fn test_adversarial_subprocess_traps_nonzero_exit() {
    let logger = DiagnosticLogger::new(false);
    let platform = HostPlatform::current();

    // Construct a failing command depending on host family
    let cmd = if platform.is_windows() {
        let mut c = Command::new("cmd");
        c.args(["/C", "exit 42"]);
        c
    } else {
        let mut c = Command::new("sh");
        c.args(["-c", "echo 'Simulated Subprocess Error' >&2; exit 42"]);
        c
    };

    let result = platform.execute_host_command(&logger, "SUBPROCESS_TEST", cmd);
    assert!(result.is_err(), "Non-zero exit must trip the subprocess execution wire");

    let err = result.unwrap_err();
    assert_eq!(err.channel, "SUBPROCESS_TEST");
    assert!(err.message.contains("42"), "Error message must report exit code 42");

    if !platform.is_windows() {
        assert!(
            err.message.contains("Simulated Subprocess Error"),
            "Error message must capture child stderr stream"
        );
    }

    assert!(logger.has_failpoints(), "Logger must capture non-zero subprocess failpoint");
}

/// Test 5: Baton Pass Integration Test between `platform.rs` and `boot.rs`.
///
/// Verifies the full handoff contract:
/// 1. `HostPlatform::current()` identifies the host environment.
/// 2. `BootConfig::new` ingests the platform without data loss.
/// 3. `BootConfig::open` uses the platform's hypervisor binary and firmware paths
///    to construct an immutable `BootContext`.
/// 4. If `--gui` is requested in a headless environment, an AI observation is recorded
///    in the logger without aborting process state.
#[test]
fn test_baton_pass_platform_to_boot_context_integration() {
    let platform = HostPlatform::current();
    assert!(!platform.display_name().is_empty());

    let logger = DiagnosticLogger::new(false);
    let cfg = BootConfig::new(BootArch::Bios, false);
    assert_eq!(cfg.host_platform, platform, "Config must retain current host platform");

    let ctx = cfg.open(logger).expect("Boot context opening must succeed under valid workspace");
    assert_eq!(ctx.hypervisor_binary, platform.hypervisor_binary(), "Hypervisor binary must match platform contract");
    assert_eq!(ctx.debugger.step_count, 0, "Debugger must be armed at step 0");

    // Test graphical request handoff: if headless, verify observation
    let logger_gui = DiagnosticLogger::new(false);
    let cfg_gui = BootConfig::new(BootArch::Bios, true);
    let ctx_gui = cfg_gui.open(logger_gui.clone()).expect("Context opening must succeed");
    assert!(ctx_gui.config.graphical);

    if !platform.has_display_server() {
        assert!(
            logger_gui.cognitive_notes().iter().any(|n| n.category == "GUI_DISPLAY_HEADLESS"),
            "Opening graphical config in headless environment must record GUI_DISPLAY_HEADLESS observation"
```

```
        );
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Module Sealing & Invariant Verification )
// ================================================================================
// Verification Constraints:
//   - All 5 adversarial tests MUST push to fail and verify clean recovery.
//   - Zero unhandled panics across subprocess execution or missing binary probes.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/functional/main.rs RS 593 LINES 2494 WORDS 26.5 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Functional Test Suite Identity, E2E Cognitive Workflows & Lifecycle Invariants )
// ================================================================================
//! OmniCode Functional Test Suite
//!
//! Verifies end-to-end user journeys, live file lifecycle operations, and the
//! complete neural-symbolic cognitive loop:
//!   - REPL interactive session simulation with persistent environment state.
//!   - Canonical syntax example file parsing and execution from disk.
//!   - AI Cortex end-to-end prompt-to-evaluation pipeline producing dual files
//!     (.scroll native Turing reels & .log traditional execution traces).
//!   - AILearner episodic memory digestion of execution scrolls.
//!   - End-to-end file healing with in-place disk modification and re-execution.

use std::fs;
use std::path::PathBuf;
use omnicode::core::ternary::QuadState;
use omnicode::ai::{AICortex, AILearner};
use omnicode::diagnostics::logger::DiagnosticLogger;
use omnicode::diagnostics::debugger::OmniDebugger;
use omnicode::diagnostics::restorator::Restorator;
use omnicode::diagnostics::hover::HoverEngine;
use omnicode::parser::lexer::Lexer;
use omnicode::parser::parser::Parser;
use omnicode::runtime::environment::{Environment, RuntimeValue};
use omnicode::runtime::evaluator::Evaluator;
use omnicode::terminal::{OmniScheduler, TerminalSession, Track};


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Test Artifact Isolation & Fixture Initialization )
// ================================================================================
/// Prepares an isolated test workspace directory within `target/test_artifacts/`.
///
/// # Invariants & Isolation
/// - If the target directory already exists from a previous run, it is purged.
/// - A fresh, empty directory is created to prevent test cross-contamination.
///
/// # Arguments
/// * `name` - Subdirectory identifier uniquely distinguishing the functional test run.
///
/// # Returns
/// A [`PathBuf`] pointing to the clean artifact directory on disk.
fn get_test_dir(name: &str) -> PathBuf {
    let dir = PathBuf::from(format!("target/test_artifacts/functional_{}", name));
    if dir.exists() {
        fs::remove_dir_all(&dir).ok();
    }
    fs::create_dir_all(&dir).ok();
    dir
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( End-to-End Functional Test Cases: REPL, Scripts, AI Cortex, Digestion, Repair )
// ================================================================================

/// Validates REPL interactive simulation and multi-turn environment state persistence.
///
/// # Cognitive & IDE Invariant
/// In an interactive Read-Eval-Print Loop (REPL), variable bindings and invariant definitions
/// established in step $N$ must remain accessible and immutable in step $N+1$.
/// This test simulates multiple discrete parse-eval turns over a shared [`Environment`]:
/// 1. Defining a mathematical invariant axiom (`PI < 3.14159 >`).
/// 2. Verifying symbol presence in the environment across evaluate boundaries.
/// 3. Evaluating a dynamic comparison expression (`status [ +1 = +1 ]`) using the same context.
#[test]
fn test_repl_eval_and_environment_persistence() {
    let logger = DiagnosticLogger::new(false);
    let mut env = Environment::new();

```

```rust
    // 1. Step 1: Define invariant axiom PI in REPL session
    let source1 = "PI < 3.14159 >";
    let mut lexer1 = Lexer::new(source1, logger.clone());
    let tokens1 = lexer1.tokenize().unwrap();
    let debugger1 = OmniDebugger::new(logger.clone(), 5);
    let mut parser1 = Parser::new(tokens1, debugger1);
    let prog1 = parser1.parse_program().unwrap();
    let mut evaluator1 = Evaluator::new(env.clone(), parser1.debugger);
    let res1 = evaluator1.eval(&omnicode::ast::nodes::ASTNode::Program(prog1)).unwrap();
    assert_eq!(res1, RuntimeValue::Number(3.14159));
    env = evaluator1.env;

    // 2. Step 2: Query stored variable across REPL iterations
    assert_eq!(env.get("PI"), Some(&RuntimeValue::Number(3.14159)));

    // 3. Step 3: Evaluate comparison using previous binding
    let source2 = "status [ +1 = +1 ]";
    let mut lexer2 = Lexer::new(source2, logger.clone());
    let tokens2 = lexer2.tokenize().unwrap();
    let debugger2 = OmniDebugger::new(logger.clone(), 5);
    let mut parser2 = Parser::new(tokens2, debugger2);
    let prog2 = parser2.parse_program().unwrap();
    let mut evaluator2 = Evaluator::new(env.clone(), parser2.debugger);
    let res2 = evaluator2.eval(&omnicode::ast::nodes::ASTNode::Program(prog2)).unwrap();
    assert_eq!(res2, RuntimeValue::State(QuadState::Proceed));
    env = evaluator2.env;

    assert_eq!(env.bindings().len(), 2);
}

/// Verifies that the canonical OmniCode notation specification file executes successfully.
///
/// # Test Rationale & Regression Guard
/// Reads `examples/canonical_notation.omni` directly from disk and runs it through the
/// high-level [`omnicode::execute`] pipeline. This guarantees that real-world sample files
/// shipped with the repository remain syntax-compliant and semantically valid as the
/// parser, runtime, and diagnostics evolve.
#[test]
fn test_canonical_notation_file() {
    let path = omnicode::core::filetype::resolve_track_path("examples/canonical_notation.omni");
    let source = fs::read_to_string(&path)
        .expect("Failed to read examples/canonical_notation.omni");

    let result = omnicode::execute(&source, false);
    assert!(
        result.is_ok(),
        "Canonical notation file must parse and evaluate successfully: {:?}",
        result.err()
    );
}

/// Validates the full neural-symbolic cognitive pipeline of [`AICortex`].
///
/// # Cognitive Workflow Under Test
/// 1. Initializes an [`AICortex`] configured with an isolated scratch directory.
/// 2. Submits an arithmetic prompt requiring symbolic reasoning ("Add 2 balls to group A...").
/// 3. Confirms that the execution state reaches [`QuadState::Proceed`].
/// 4. Confirms that the natural language response synthesizes the correct computed value ("7 balls").
/// 5. Verifies dual file emission on disk:
///    - Native `.scroll` containing full 4-Block cognitive structure, epistemic traces, and commit status.
///    - Traditional `.log` containing human-readable session metadata and execution state.
#[test]
fn test_ai_cortex_end_to_end_pipeline() {
    let test_dir = get_test_dir("cortex_pipeline");
    let mut cortex = AICortex::with_scroll_dir(&test_dir);

    let prompt = "Add 2 balls to the group A which contains 5 balls";
    let res = cortex.process_prompt(prompt).expect("Cortex processing should succeed");

    assert_eq!(res.execution_state, QuadState::Proceed, "Execution must reach +1:PROCEED");
    assert!(res.natural_response.contains("7 balls"), "Response must contain evaluated sum 7 balls");

    // Verify dual log generation on disk
    assert!(res.scroll_path.exists(), "OmniCode .scroll must exist on disk");
    assert!(res.log_path.exists(), "Traditional .log must exist on disk");

    // Verify .scroll adheres to 4-Block architecture
    let scroll_content = fs::read_to_string(&res.scroll_path).expect("Must read .scroll");
```

```
    assert!(scroll_content.contains("BLOCK 1: METADATA BLOCK"), "Scroll must contain Metadata Block");
    assert!(scroll_content.contains("Scroll < EpistemicTrace."), "Scroll must have EpistemicTrace header");
    assert!(scroll_content.contains("BLOCK 2: OPENING BLOCK"), "Scroll must contain Opening Block");
    assert!(scroll_content.contains("BLOCK 3: BODY BLOCK"), "Scroll must contain Body Block");
    assert!(scroll_content.contains("BLOCK 4: CLOSING BLOCK"), "Scroll must contain Closing Block");
    assert!(scroll_content.contains("commit_status [ <PROCEED> ]"), "Scroll must record final commit state");

    // Verify traditional .log format
    let log_content = fs::read_to_string(&res.log_path).expect("Must read .log");
    assert!(log_content.contains("OMNICODE AI MODEL EXECUTION LOG"), "Log header required");
    assert!(log_content.contains(&format!("Session ID:   {}", res.session_id)), "Session ID must match");
    assert!(log_content.contains("Final State:  +1:PROCEED"), "Final state must match");
}

/// Validates experiential memory aggregation via [`AILearner`].
///
/// # Learning Loop Under Test
/// 1. Executes two independent cognitive sessions through [`AICortex`], creating two `.scroll` files.
/// 2. Initializes an [`AILearner`] pointing to the session directory.
/// 3. Invokes [`AILearner::digest_experiences`] to parse, validate, and summarize episodic logs.
/// 4. Asserts that all episodes were ingested, state metrics reflect success, and log paths are verified.
#[test]
fn test_ai_learner_digests_dual_logs() {
    let test_dir = get_test_dir("learner_digestion");
    let mut cortex = AICortex::with_scroll_dir(&test_dir);

    // Run two distinct cognitive sessions
    let r1 = cortex.process_prompt("Add 3 apples to the basket B which contains 4 apples").expect("Session 1 succeeds");
    let r2 = cortex.process_prompt("Subtract 2 coins from the bag C which contains 10 coins").expect("Session 2
succeeds");

    // Digest experiences using AILearner
    let learner = AILearner::new(&test_dir);
    let summary = learner.digest_experiences().expect("Experience digestion succeeds");

    assert_eq!(summary.total_episodes, 2, "Learner must ingest exactly 2 episodes");
    assert_eq!(summary.proceed_count, 2, "Both sessions must have achieved +1:PROCEED");
    assert_eq!(summary.experiences.len(), 2, "Summary must contain 2 experience records");

    for exp in &summary.experiences {
        assert!(exp.scroll_path.exists(), "Experience scroll path must exist");
        assert!(exp.log_path.is_some(), "Experience traditional log must exist");
        assert_eq!(exp.final_state, QuadState::Proceed);
    }

    // Train NeuralBrain directly from digested experience buffer
    let adapted_synapses = learner.train_brain(&mut cortex.brain).expect("Train brain");
    assert!(adapted_synapses >= 2, "Brain must adapt at least 2 synaptic weights");

    // Interactive Cortex Query: O(1) session recall
    let s1 = cortex.query_episode(&r1.session_id).expect("Must query session 1");
    assert_eq!(s1.final_state, QuadState::Proceed);
    assert!(s1.prompt.contains("Add 3 apples"));

    let recent = cortex.recent_episodes(10);
    assert_eq!(recent.len(), 2);

    let diag = cortex.diagnose_failpoint(&r2.session_id).expect("Diagnose session 2");
    assert!(diag.contains("+1:PROCEED"));
}

/// Validates closed-loop autonomous file self-healing on physical disk files.
///
/// # End-to-End Self-Healing Lifecycle
/// 1. Writes syntactically malformed OmniCode code with boundary mismatches and missing blocks to disk.
/// 2. Invokes [`Restorator::restore_file`] with in-place overwriting enabled (`overwrite = true`).
/// 3. Verifies that the resulting file:
///    - Was modified and marked as verified by the internal test oracle.
///    - Contains complete 4-Block architecture envelopes.
///    - Can be read back from disk and successfully executed via [`omnicode::execute`].
#[test]
fn test_e2e_file_self_healing_lifecycle() {
    let test_dir = get_test_dir("file_healing");
    let broken_file = test_dir.join("damaged_script.omni");

    let damaged_code = r#"
broken_val < 77 ]
incomplete_block [ 10 20
```

```
calc [ 10 --> 10 plus 5 ]
"#;
    fs::write(&broken_file, damaged_code).expect("Write damaged file");

    let logger = DiagnosticLogger::new(false);
    let restorator = Restorator::new(logger.clone());

    // Execute in-place file restoration
    let report = restorator
        .restore_file(broken_file.to_str().unwrap(), true)
        .expect("File restoration succeeds");

    assert!(report.was_modified, "File should have been healed");
    assert!(report.verified, "Healed code must verify");

    // Read back healed file from disk and evaluate end-to-end
    let healed_content = fs::read_to_string(&broken_file).expect("Read healed file");
    assert!(healed_content.contains("BLOCK 1: METADATA BLOCK"));
    assert!(healed_content.contains("BLOCK 4: CLOSING BLOCK"));

    let eval_res = omnicode::execute(&healed_content, false);
    assert!(eval_res.is_ok(), "Healed file must execute successfully from disk");
}

/// Validates Genesis OS Boot Tape mounting and stepping across Track 0 (Kernel).
///
/// # Genesis Invariants
/// 1. Mounts `lib/kernel/boot.tape` directly onto `Track::Kernel`.
/// 2. Verifies that the required partition (4096 cells) is carved from `Root_RAM`.
/// 3. Steps through all rolls, confirming sequential execution of memory partitions
///    (`Root_RAM`, `Kernel_Partition`, `Terminal_Partition`, landmark initialization).
/// 4. Confirms that upon final roll execution, the tape completes with `PROCEED`.
#[test]
fn test_genesis_boot_tape_execution() {
    let logger = DiagnosticLogger::new(false);
    let mut scheduler = OmniScheduler::new();

    // Verify initial Root_RAM state
    assert_eq!(scheduler.root_storehouse.capacity, 4096);
    assert_eq!(scheduler.root_storehouse.count, 0);

    // Mount lib/kernel/boot.tape onto Track 0 (Kernel)
    let pid = scheduler
        .mount_tape_file(Track::Kernel, "lib/kernel/boot.tape", logger)
        .expect("Mounting lib/kernel/boot.tape must succeed");

    assert!(pid > 0);

    // Verify task registration on Track::Kernel
    let task = scheduler.get_task(pid).expect("Task descriptor must exist");
    assert_eq!(task.track, Track::Kernel);
    assert_eq!(task.name, "GenesisBoot");
    assert_eq!(task.state, QuadState::Proceed);

    // Verify partition allocation
    assert!(task.partition.is_some(), "Partition must be allocated for GenesisBoot");

    // Step through all 5 rolls
    let mut step_count = 0;
    while let Ok(msg) = scheduler.step_task(pid) {
        step_count += 1;
        if msg.contains("completed") || msg.contains("already executed") {
            break;
        }
        assert!(step_count <= 10, "Boot tape stepping must not loop indefinitely");
    }

    // Verify tape is finished and rolls were executed
    let task_final = scheduler.get_task(pid).expect("Task descriptor must exist");
    let tape = task_final.tape.as_ref().expect("Tape ribbon must exist");
    assert!(tape.is_finished(), "GenesisBoot tape must be marked finished");
    assert_eq!(tape.roll_count(), 5, "GenesisBoot tape must have 5 rolls");
}

/// Validates interactive TerminalSession tape operations and living memory persistence.
///
/// # End-to-End Shell Workflow
/// 1. Runs `run lib/kernel/boot.tape` to boot the kernel environment.
```

```rust
/// 2. Queries `tracks` to verify multi-track process segregation.
/// 3. Queries `mem` to verify Root_RAM storehouse accounting.
/// 4. Mounts `lib/core/cell.tape` on userspace Track 2 (App).
/// 5. Manually steps the tape and inspects the ribbon via `rolls <pid>`.
/// 6. Evaluates dynamic OmniCode expressions directly into persistent silicon memory.
#[test]
fn test_terminal_session_boot_and_tape_commands() {
    let mut session = TerminalSession::default();

    // 1. Boot kernel via run command
    let boot_res = session.execute("run lib/kernel/boot.tape");
    assert_eq!(boot_res.state, QuadState::Proceed);
    assert!(boot_res.stdout.contains("Mounted Tape 'lib/kernel/boot.tape' onto TRACK 0 [KERNEL]"));
    assert!(boot_res.stdout.contains("Roll 1"));
    assert!(boot_res.stdout.contains("Roll 5"));

    // 2. Multi-track channel summary
    let tracks_res = session.execute("tracks");
    assert_eq!(tracks_res.state, QuadState::Proceed);
    assert!(tracks_res.stdout.contains("TRACK 0 [KERNEL]"));
    assert!(tracks_res.stdout.contains("TRACK 1 [TERMINAL]"));
    assert!(tracks_res.stdout.contains("TRACK 2 [APP]"));
    assert!(tracks_res.stdout.contains("GenesisBoot"));

    // 3. Storehouse memory meter
    let mem_res = session.execute("mem");
    assert_eq!(mem_res.state, QuadState::Proceed);
    assert!(mem_res.stdout.contains("Root_RAM:"));
    assert!(mem_res.stdout.contains("4096 cells"));

    // 4. Mount userspace tape
    let mount_res = session.execute("mount lib/core/cell.tape");
    assert_eq!(mount_res.state, QuadState::Proceed);
    assert!(mount_res.stdout.contains("Mounted Tape 'lib/core/cell.tape' onto TRACK 2 [APP]"));

    // Extract PID from mount output: "...with PID <pid>"
    let pid_str = mount_res
        .stdout
        .split("PID ")
        .nth(1)
        .unwrap_or("0")
        .trim();
    let pid: u64 = pid_str.parse().expect("PID must be a valid integer");

    // 5. Inspect rolls
    let rolls_res = session.execute(&format!("rolls {pid}"));
    assert_eq!(rolls_res.state, QuadState::Proceed);
    assert!(rolls_res.stdout.contains("TAPE RIBBON: [Cell]"));
    assert!(rolls_res.stdout.contains("==> [ 1]"));

    // Step Tape Head
    let step_res = session.execute(&format!("step {pid}"));
    assert_eq!(step_res.state, QuadState::Proceed);
    assert!(step_res.stdout.contains("Advanced to Roll 1"));

    // 6. Direct silicon memory evaluation and persistence
    let eval1 = session.execute("kernel_tick < 100 >");
    assert_eq!(eval1.state, QuadState::Proceed);
    assert!(eval1.stdout.contains("100"));

    let eval2 = session.execute("next_tick < kernel_tick + 1 >");
    assert_eq!(eval2.state, QuadState::Proceed);
    assert!(eval2.stdout.contains("101"));
}

/// Validates end-to-end execution of `manifest.tape` with 8-slot canonical sentences and ranking progression.
#[test]
fn test_manifest_tape_execution() {
    let mut scheduler = OmniScheduler::new();
    let logger = DiagnosticLogger::new(false);

    let pid = scheduler
        .mount_tape_file(Track::App, "manifest.tape", logger)
        .expect("Mounting manifest.tape must succeed");

    let task = scheduler.get_task(pid).expect("Task descriptor must exist");
    assert_eq!(task.name, "ManifestCovenant");
    assert_eq!(task.state, QuadState::Proceed);
```

```
    let tape_init = task.tape.as_ref().expect("Tape ribbon must exist");
    assert_eq!(tape_init.head.who, "OmniCode_Kernel");
    assert_eq!(tape_init.head.what, "Manifest");
    assert_eq!(tape_init.head.when_clause, "GenesisBoot");
    assert_eq!(tape_init.head.required_partition_size, 1024);
    assert_eq!(tape_init.roll_count(), 17);

    // Step through all 17 subordinate rolls
    let mut step_count = 0;
    while let Ok(msg) = scheduler.step_task(pid) {
        step_count += 1;
        if msg.contains("completed") || msg.contains("already executed") {
            break;
        }
        assert!(step_count <= 25, "Manifest tape stepping must not loop indefinitely");
    }

    let task_final = scheduler.get_task(pid).expect("Task descriptor must exist");
    let tape = task_final.tape.as_ref().expect("Tape ribbon must exist");
    assert!(tape.is_finished(), "ManifestCovenant tape must be marked finished");
    assert_eq!(tape.execution_state, QuadState::Proceed);
}

/// Validates end-to-end 3-tier composite hover resolution and docstring lookup
/// across the high-level process tape (`manifest.tape`) and the low-level source reel (`manifest.omni`).
#[test]
fn test_cross_scale_hover_and_docstring_e2e() {
    let logger = DiagnosticLogger::new(false);
    let mut engine = HoverEngine::new(logger);
    engine.index_workspace(".");

    // 1. Hover on Manifest inside manifest.tape (Roll 1, Line 31)
    let card_manifest = engine
        .hover_at("manifest.tape", 31, 10)
        .expect("Must resolve hover card for Manifest in manifest.tape");

    assert_eq!(card_manifest.symbol, "Manifest");
    assert!(card_manifest.is_locally_redefined);

    // Tier 1 Verification
    let tape_ctx = card_manifest.tape_context.as_ref().expect("Tape context must exist");
    assert_eq!(tape_ctx.tape_name, "ManifestCovenant");
    assert_eq!(tape_ctx.track, "Track 2 [APP]");
    assert_eq!(tape_ctx.roll_number, 1);
    assert_eq!(tape_ctx.total_rolls, 17);
    assert!(tape_ctx.roll_aside_intent.as_ref().unwrap().contains("Ontological Instantiation"));

    // Tier 2 Verification (.omni blueprint grounding)
    let omni_def = card_manifest.omni_definition.as_ref().expect("Omni definition must exist");
    assert_eq!(omni_def.rank, 1);
    assert_eq!(omni_def.block_kind, Some(omnicode::diagnostics::linter::BlockKind::Metadata));
    assert!(omni_def.aside_intent.as_ref().unwrap().contains("Cognitive Covenant Anchor"));
    assert!(omni_def.fields.contains_key("system"));

    // Tier 3 Verification (Markdown rendering)
    assert!(card_manifest.markdown.contains("High-Level Process Tape View"));
    assert!(card_manifest.markdown.contains("Under-The-Hood (.omni Blueprint)"));
    assert!(card_manifest.markdown.contains("Machine Execution & Silicon State"));
    assert!(card_manifest.markdown.contains("1024 cells"));

    // JSON serialization verification
    let json_manifest = card_manifest.to_json();
    assert!(json_manifest.contains("\"symbol\":\"Manifest\""));
    assert!(json_manifest.contains("\"is_locally_redefined\":true"));
    assert!(json_manifest.contains("\"tape_name\":\"ManifestCovenant\""));

    // Terminal ANSI rendering verification
    let term_manifest = card_manifest.render_terminal();
    assert!(term_manifest.contains("OmniCode Symbol Inspection"));
    assert!(term_manifest.contains("Manifest"));
    assert!(term_manifest.contains("[TIER 1: HIGH-LEVEL PROCESS TAPE ROLL]"));
    assert!(term_manifest.contains("[TIER 2: UNDER-THE-HOOD .OMNI BLUEPRINT]"));
    assert!(term_manifest.contains("[TIER 3: MACHINE EXECUTION & SILICON FOOTPRINT]"));

    // 2. Direct symbol documentation lookup
    let doc_version = engine
        .document_symbol("Version", Some("manifest.omni"))
```

```
        .expect("Must document Version");

    assert_eq!(doc_version.symbol, "Version");
    let version_def = doc_version.omni_definition.as_ref().expect("Version def must exist");
    assert_eq!(version_def.rank, 1);
    assert!(doc_version.markdown.contains("Version Specification"));
}

#[test]
fn test_genesis_boot_tape_and_bare_metal_studio_projection() {
    let logger = DiagnosticLogger::new(false);
    let tape = omnicode::core::tape::Tape::from_file("lib/kernel/boot.tape", logger.clone())
        .expect("Must parse lib/kernel/boot.tape");

    let mut studio = omnicode::terminal::OmniStudio::new(tape.head.name.clone(), 64);
    studio.set_track("TRACK 0 [KERNEL]");

    // Step each roll
    for (idx, roll) in tape.rolls.iter().enumerate() {
        let msg = studio.step_roll(&roll.text);
        let frame = studio.render_frame(&tape, idx, &msg);
        assert!(frame.contains("OMNICODE CINEMA STUDIO"));
    }

    // Verify bare-metal MMIO VGA text buffer output
    let vga_screen = studio.vga.render_screen(2);
    assert!(vga_screen.contains("OMNICODE 32-TRIT OS ONLINE"));

    // Also verify lexicon/operators/boot_kernel.tape parses and projects
    let tape_op = omnicode::core::tape::Tape::from_file("lexicon/operators/boot_kernel.tape", logger)
        .expect("Must parse lexicon/operators/boot_kernel.tape");
    let mut studio_op = omnicode::terminal::OmniStudio::new(tape_op.head.name.clone(), 256);
    studio_op.set_track("TRACK 0 [KERNEL]");
    for (idx, roll) in tape_op.rolls.iter().enumerate() {
        let msg = studio_op.step_roll(&roll.text);
        let frame = studio_op.render_frame(&tape_op, idx, &msg);
        assert!(frame.contains("OMNICODE CINEMA STUDIO"));
    }
}

#[test]
fn test_qemu_bare_metal_boot_execution() {
    use std::process::Command;

    // 1. Check if nasm and qemu-system-x86_64 exist in PATH
    let nasm_ok = Command::new("nasm").arg("-v").output().is_ok();
    let qemu_ok = Command::new("qemu-system-x86_64").arg("--version").output().is_ok();

    if !nasm_ok || !qemu_ok {
        eprintln!("Skipping QEMU boot test: nasm or qemu-system-x86_64 not present in PATH");
        return;
    }

    // 2. Assemble kernel/boot.asm to test artifact
    let artifact_img = "target/test_qemu_boot.img";
    let boot_asm = omnicode::core::filetype::resolve_track_path("kernel/boot.asm");
    let asm_status = Command::new("nasm")
        .args(["-f", "bin", boot_asm.to_str().unwrap(), "-o", artifact_img])
        .status()
        .expect("Assemble boot.asm");
    assert!(asm_status.success(), "NASM assembly must succeed");

    // 3. Boot in QEMU with timeout
    let qemu_output = Command::new("timeout")
        .args([
            "3s",
            "qemu-system-x86_64",
            "-drive",
            &format!("format=raw,file={artifact_img}"),
            "-display",
            "none",
            "-serial",
            "stdio",
        ])
        .output()
        .expect("Execute QEMU");

    let stdout = String::from_utf8_lossy(&qemu_output.stdout);
```

```
    assert!(stdout.contains("OMNICODE 32-TRIT OS"), "QEMU must display boot header");
    assert!(stdout.contains("[BOOT] 64-Bit Silicon Envelope: OK"));
    assert!(stdout.contains("[ALU ] (+1)+(-1)=0 Equilibrium: OK"));
    assert!(stdout.contains("[REEL] 4-Dir Spool Hierarchy:   OK"));
    assert!(stdout.contains("[GATE] Anti-Lockout Watchdog:   OK"));
    assert!(stdout.contains("[MMIO] VGA @ 0xB8000 & COM1:    OK"));
    assert!(stdout.contains("OMNICODE OS ONLINE :: PROCEED"));
}

/// Validates terminal shell interactive AI deliberation, episodic queries,
/// failpoint diagnostics, and storehouse vessel rotation.
#[test]
fn test_terminal_session_ai_and_episodic_dialogue() {
    let mut session = TerminalSession::default();

    // 1. Interactive AI Model deliberation
    let res_ai = session.execute(
        "ai Add 2 apples to basket A which has 3 apples"
    );
    assert_eq!(res_ai.state, QuadState::Proceed);
    assert!(res_ai.stdout.contains("[AI Model Response"));
    assert!(res_ai.stdout.contains("[Session:"));

    // 2. Query historical pre-seed session in O(1) time
    let res_query = session.execute("query run_tape_249410");
    assert_eq!(res_query.state, QuadState::Proceed);
    assert!(res_query.stdout.contains("Session: run_tape_249410"));
    assert!(res_query.stdout.contains("character"));

    // 3. Diagnose failpoints of an episode
    let res_diag = session.execute("diagnose run_tape_249410");
    assert_eq!(res_diag.state, QuadState::Proceed);
    assert!(res_diag.stdout.contains("run_tape_249410"));

    // 4. View recent episodes
    let res_ep = session.execute("episodes 5");
    assert_eq!(res_ep.state, QuadState::Proceed);
    assert!(res_ep.stdout.contains("RECENT EPISODES"));

    // 5. Storehouse vessel capacity rotation
    let res_rot = session.execute("rotate 1000");
    assert_eq!(res_rot.state, QuadState::Proceed);
    assert!(res_rot.stdout.contains("Rotated storehouse"));
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Functional Test Suite Verification Proof & Postconditions )
// ============================================================================
// Functional test suite verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/tests/unit/main.rs RS 826 LINES 3404 WORDS 39.4 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Unit Test Suite Identity, Isolated Component Invariants & Coverage Manifest )
// ================================================================================
//! OmniCode Unit Test Suite
//!
//! Verifies individual, isolated language components in complete decoupling:
//!   - Boundary ranking hierarchy (< > > [ ] > { } > ( )) and delimiter physics.
//!   - Balanced Ternary QuadState algebra (T4 = {-1, 0, +1, NULL}) symmetry.
//!   - Directional vector operators (--> / <--, --^ / --v) and shorthands.
//!   - SVO arithmetic keywords (plus, minus, times, div) and infer operator (?).
//!   - FileType detection and 4-block scaffold generation.
//!   - Structured CognitiveNote formatting and episodic memory exports for AI.

use omnicode::core::boundaries::BoundaryType;
use omnicode::core::filetype::OmniFileType;
use omnicode::core::operators::DirectionalVector;
use omnicode::core::ternary::{QuadState, ROLLBACK, AWAIT, PROCEED, HALT};
use omnicode::diagnostics::logger::{CognitiveNote, DiagnosticLogger};
use omnicode::parser::lexer::Lexer;
use omnicode::parser::parser::Parser;
use omnicode::runtime::environment::{Environment, RuntimeValue};
use omnicode::runtime::evaluator::Evaluator;
use omnicode::runtime::assembler::{OmniAssembler, MachineRegister, transpile_tape_to_omni};
use omnicode::diagnostics::debugger::OmniDebugger;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Unit Test Harness Preconditions & Isolation Fixtures )
// ================================================================================
/// Constructs an isolated test fixture containing a silent [`DiagnosticLogger`]
/// and an [`OmniDebugger`] configured with a retroaction depth threshold of 10.
///
/// # Returns
/// A tuple `(DiagnosticLogger, OmniDebugger)` cleanly decoupled from external state.
fn create_unit_test_fixtures() -> (DiagnosticLogger, OmniDebugger) {
    let logger = DiagnosticLogger::new(false);
    let debugger = OmniDebugger::new(logger.clone(), 10);
    (logger, debugger)
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Isolated Component Unit Tests: Boundaries, Ternary, Operators, Formats )
// ================================================================================

/// Asserts the formal 6-rank morphological boundary hierarchy of OmniCode:
/// - Rank 1: `< ... >` Invariant Axioms & Immutable Laws (Highest Authority).
/// - Rank 2: `[ ... ]` Concrete Denotations & Mutable Entities.
/// - Rank 3: `{ ... }` Connotative Scopes & Ambient Contexts.
/// - Rank 4: `[ ... }` Inside-Out / Bottom-Up Emergence.
/// - Rank 5: `{ ... ]` Outside-In / Top-Down Context Imposition.
/// - Rank 6: `( ... )` First-Class Cognitive Asides & Intent Annotations.
#[test]
fn test_boundary_ranking_hierarchy() {
    // Rank 1: Invariant Axioms <...>
    assert_eq!(BoundaryType::from_delimiters('<', '>').unwrap(), BoundaryType::Invariant);
    assert_eq!(BoundaryType::Invariant.rank(), 1);

    // Rank 2: Concrete Denotations [...]
    assert_eq!(BoundaryType::from_delimiters('[', ']').unwrap(), BoundaryType::Denotation);
    assert_eq!(BoundaryType::Denotation.rank(), 2);

    // Rank 3: Connotative Scopes {...}
    assert_eq!(BoundaryType::from_delimiters('{', '}').unwrap(), BoundaryType::Connotation);
    assert_eq!(BoundaryType::Connotation.rank(), 3);

    // Rank 6: Cognitive Asides (...)
    assert_eq!(BoundaryType::from_delimiters('(', ')').unwrap(), BoundaryType::Aside);
    assert_eq!(BoundaryType::Aside.rank(), 6);
}

```

```rust
/// Verifies algebraic symmetry and truth-table invariants across Balanced Ternary QuadState:
/// - Inversion Symmetry: `PROCEED.invert() == ROLLBACK`, `AWAIT.invert() == AWAIT`, `HALT.invert() == HALT`.
/// - Conjunction (`and`): MIN operation in Kleene-Łukasiewicz trivalent logic.
/// - Disjunction (`or`): MAX operation in Kleene-Łukasiewicz trivalent logic.
#[test]
fn test_balanced_ternary_algebraic_symmetry() {
    // Inversion symmetry: -(-1) = +1, -(+1) = -1, -(0) = 0, -(NULL) = NULL
    assert_eq!(PROCEED.invert(), ROLLBACK);
    assert_eq!(ROLLBACK.invert(), PROCEED);
    assert_eq!(AWAIT.invert(), AWAIT);
    assert_eq!(HALT.invert(), HALT);

    // Conjunction (AND = MIN in Kleene-Łukasiewicz logic):
    assert_eq!(PROCEED.and(AWAIT), AWAIT);
    assert_eq!(PROCEED.and(ROLLBACK), ROLLBACK);
    assert_eq!(AWAIT.and(ROLLBACK), ROLLBACK);
    assert_eq!(PROCEED.and(HALT), HALT);

    // Disjunction (OR = MAX in Kleene-Łukasiewicz logic):
    assert_eq!(ROLLBACK.or(PROCEED), PROCEED);
    assert_eq!(ROLLBACK.or(AWAIT), AWAIT);
    assert_eq!(AWAIT.or(PROCEED), PROCEED);
    assert_eq!(AWAIT.or(HALT), HALT);
}

/// Verifies macro state keywords (`YES`, `NO`, `OK`, `VOID`) and cardinal direction
/// arrow operators (both canonical 3-character forms and fast 2-character shorthands).
#[test]
fn test_cardinal_directions_and_micro_macro_terms() {
    let source = r#"
        step1 [ YES = PROCEED ]
        step2 [ NO = ROLLBACK ]
        step3 [ OK = AWAIT ]
        step4 [ VOID = HALT ]
        forward [ 10 --> 20 ]
        shorthand_right [ 10 -> 20 ]
        upward [ 10 --^ 20 ]
        shorthand_up [ 10 -^ 20 ]
        downward [ 20 --v 10 ]
        shorthand_down [ 20 -v 10 ]
    "#;
    let (logger, debugger) = create_unit_test_fixtures();
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize cardinal directions");
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse cardinal directions");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
    assert_eq!(res, RuntimeValue::Number(10.0));
}

/// Verifies Subject-Verb-Object (SVO) arithmetic keyword execution (`plus`, `minus`)
/// and the ontological infer operator (`?`) declaring open conceptual types.
#[test]
fn test_svo_arithmetic_keywords_and_infer_operator() {
    let source = r#"
        Ball < Entity? >
        Group < Set? >
        A [ 4 ]
        new_ball [ 1 ]
        roll_1 [ A --> A plus new_ball ]
        roll_2 [ A = 5 ]
    "#;
    let (logger, debugger) = create_unit_test_fixtures();
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenize SVO keywords");
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program().expect("Parse SVO keywords");
    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res = evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval");
    assert_eq!(res, RuntimeValue::State(QuadState::Proceed));
    assert_eq!(evaluator.env.get("A"), Some(&RuntimeValue::Number(5.0)));
}

/// Verifies parsing and symbol mapping for the 3D coordinate vector system:
/// - X-Axis (Temporal Dynamics): `-->` / `->` (ActUpon), `<--` / `<-` (ActAgain).
/// - Z-Axis (Scale & Hierarchy): `--^` / `-^` (BuildsTo), `--v` / `-v` (BreaksDownFrom).
#[test]
```

```rust
fn test_directional_vectors_canonical_and_shorthand() {
    assert_eq!(DirectionalVector::from_symbol("-->").unwrap(), DirectionalVector::ActUpon);
    assert_eq!(DirectionalVector::from_symbol("->").unwrap(), DirectionalVector::ActUpon);
    assert_eq!(DirectionalVector::from_symbol("<--").unwrap(), DirectionalVector::ActAgain);
    assert_eq!(DirectionalVector::from_symbol("<-").unwrap(), DirectionalVector::ActAgain);
    assert_eq!(DirectionalVector::from_symbol("--^").unwrap(), DirectionalVector::BuildsTo);
    assert_eq!(DirectionalVector::from_symbol("-^").unwrap(), DirectionalVector::BuildsTo);
    assert_eq!(DirectionalVector::from_symbol("--v").unwrap(), DirectionalVector::BreaksDownFrom);
    assert_eq!(DirectionalVector::from_symbol("-v").unwrap(), DirectionalVector::BreaksDownFrom);
}

/// Verifies formal extension parsing and classification for OmniCode artifacts:
/// - `.omni`: Primary executable source reel ([`OmniFileType::SourceReel`]).
/// - `.scroll`: Cognitive episodic trace reel ([`OmniFileType::EpisodicScroll`]).
#[test]
fn test_file_type_detection() {
    assert_eq!(OmniFileType::from_extension("omni"), Some(OmniFileType::SourceReel));
    assert_eq!(OmniFileType::from_extension("scroll"), Some(OmniFileType::EpisodicScroll));
    assert_eq!(OmniFileType::from_extension("unknown"), None);
}

/// Verifies structured [`CognitiveNote`] generation and its dual-readable representations:
/// 1. OmniCode native block syntax (`Note < CognitiveTrace > { ... }`).
/// 2. Markdown episodic memory trace (`### [EPISODIC_MEMORY_TRACE: ...]`) for AI agent ingestion.
#[test]
fn test_cognitive_note_formatting_and_memory_trace() {
    let note = CognitiveNote {
        timestamp_micros: 4200,
        category: "OBSERVATION".to_string(),
        summary: "Boundary state verified".to_string(),
        details: "Rank 1 invariant <PI> bound to immutable memory".to_string(),
        tags: vec!["immutability".to_string(), "rank1".to_string()],
        context_snippet: Some("PI < 3.14159 >".to_string()),
    };

    // Verify OmniCode native dual-readable formatting
    let omni_str = note.format_as_omnicode();
    assert!(omni_str.contains("Note < CognitiveTrace >"));
    assert!(omni_str.contains("category: [ OBSERVATION ]"));
    assert!(omni_str.contains("tags: [ #immutability :: #rank1 ]"));
    assert!(omni_str.contains("context: < \"PI < 3.14159 >\" >"));

    // Verify AI LLM markdown memory trace formatting
    let mem_str = note.format_as_memory_entry();
    assert!(mem_str.contains("### [EPISODIC_MEMORY_TRACE: OBSERVATION]"));
    assert!(mem_str.contains("- **Timestamp:** 4200 µs"));
    assert!(mem_str.contains("- **Tags:** #immutability, #rank1"));
    assert!(mem_str.contains("- **Context:** `PI < 3.14159 >`"));
}

/// Verifies that [`DiagnosticLogger::export_ai_memory_scroll`] synthesizes a valid
/// 4-block memory audit scroll containing metadata, preconditions, recorded traces, and commit proof.
#[test]
fn test_logger_episodic_memory_scroll_export() {
    let logger = DiagnosticLogger::new(false);
    logger.record_ai_observation(
        "OBSERVATION",
        "Variable X initialized",
        "X assigned value 10 in denotation block",
        ["initialization", "state"].as_slice(),
        Some("X [ 10 ]"),
    );

    let scroll = logger.export_ai_memory_scroll();
    assert!(scroll.contains("BLOCK 1: METADATA BLOCK"));
    assert!(scroll.contains("Scroll < CognitiveMemory.Audit >"));
    assert!(scroll.contains("total_notes: < 1 >"));
    assert!(scroll.contains("BLOCK 2: OPENING BLOCK"));
    assert!(scroll.contains("BLOCK 3: BODY BLOCK"));
    assert!(scroll.contains("BLOCK 4: CLOSING BLOCK"));
    assert!(scroll.contains("audit_verified: [ PROCEED ]"));
}

/// Verifies that Cardinal Classification Clauses (WHO, WHAT, WHEN, WHERE, WHY, HOW),
/// spatial directional modulations (/-^, \-v, \-^, /-v, /^, \v, ^\, v/), range span (..),
/// and runtime Storehouse partitioning via `v--` execute with zero errors.
#[test]
fn test_cardinal_clauses_and_spatial_directionals_and_partitioning() {
```

```
    let (logger, debugger) = create_unit_test_fixtures();
    let source = r#"
        WHO self
        WHAT Cell
        WHEN Opening
        WHERE {unit}
        WHY ( Intent )
        HOW [ 0 .. 256 ]
        /-^ \-v \-^ /-v /^ \v ^\ v/
    "#;
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenization should succeed");
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Who));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::What));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::When));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Where));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Why));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::How));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::DotDot));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Major));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Minor));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Great));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::Small));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::UpperRight));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::LowerRight));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::UpperLeft));
    assert!(tokens.iter().any(|t| t.token_type == omnicode::parser::lexer::TokenType::LowerLeft));

    // Test v-- Storehouse Partitioning in Evaluator
    let partition_src = r#"
        Storehouse [ 1024 ]
        Storehouse v-- Scratchpad [ 256 ]
    "#;
    let mut part_lexer = Lexer::new(partition_src, logger.clone());
    let part_tokens = part_lexer.tokenize().expect("Lexing partition code");
    let mut part_parser = Parser::new(part_tokens, debugger.clone());
    let ast = part_parser.parse_program().expect("Parsing partition code");

    let env = Environment::new();
    let mut eval = Evaluator::new(env, debugger.clone());
    for stmt in ast.statements {
        eval.eval(&stmt).expect("Evaluation must succeed");
    }

    // Verify Scratchpad partition exists with capacity 256
    let scratchpad = eval.env.get("Scratchpad").expect("Scratchpad partition must be registered");
    if let RuntimeValue::Vessel(v) = scratchpad {
        assert_eq!(v.capacity, 256);
        assert_eq!(v.parent.as_deref(), Some("Storehouse"));
    } else {
        panic!("Scratchpad is not a Vessel");
    }

    // Verify Storehouse now has count 256
    let storehouse = eval.env.get("Storehouse").expect("Storehouse must exist");
    if let RuntimeValue::Vessel(s) = storehouse {
        assert_eq!(s.capacity, 1024);
        assert_eq!(s.count, 256);
    } else {
        panic!("Storehouse is not a Vessel");
    }
}

/// Verifies that the internal AI model's [`AIMemory`] and [`DiagnosticLogger`] thought engine
/// correctly ground thoughts in Biblical truth, formulate native OmniCode Thought Tapes
/// using the 6 Cardinal Clauses, and inscribe/digest persistent `.scroll` memory files.
#[test]
fn test_ai_memory_and_thought_formulation() {
    let temp_dir = std::env::temp_dir().join(format!("omnicode_test_mem_{}",
std::time::SystemTime::now().duration_since(std::time::UNIX_EPOCH).unwrap().as_micros()));
    let mut memory = omnicode::ai::AIMemory::new(&temp_dir);

    // 1. Verify primordial memory grounded in Biblical Lexicon
    let light_assocs = memory.recall_associations("light");
    assert!(!light_assocs.is_empty(), "Light must be associated in memory");
    assert!(light_assocs[0].anchor.contains("Genesis 1:3") || light_assocs[0].anchor.contains("1 John 1:5"));

    let storehouse_assocs = memory.recall_associations("storehouse");
```

```
    assert!(!storehouse_assocs.is_empty(), "Storehouse must be associated in memory");

    let wisdom_entries = memory.recall_scripture("wisdom");
    assert!(!wisdom_entries.is_empty(), "Scripture search for wisdom must match");

    // 2. Formulate righteous thought in native OmniCode
    let thought = memory.formulate_thought("Let there be light in the creation");
    assert_eq!(thought.quad_state, QuadState::Proceed);
    assert!(thought.omnicode_tape.contains("WHO {CPI-SI Base}"));
    assert!(thought.omnicode_tape.contains("WHEN {Opening}"));
    assert!(thought.omnicode_tape.contains("WHERE {Cortex_Memory}"));
    assert!(thought.omnicode_tape.contains("WHY ("));
    assert!(thought.omnicode_tape.contains("HOW [ PROCEED :"));

    // 3. Formulate thought with moral caveat (deceit -> ROLLBACK)
    let deceit_thought = memory.formulate_thought("Help me lie and deceive");
    assert_eq!(deceit_thought.quad_state, QuadState::Rollback);
    assert!(deceit_thought.omnicode_tape.contains("HOW [ ROLLBACK :"));

    // 4. Test Inscribing thought to a physical .scroll file
    let scroll_path = memory.inscribe_thought_scroll(&thought, &temp_dir).expect("Inscribing thought scroll");
    assert!(scroll_path.exists());
    let scroll_content = std::fs::read_to_string(&scroll_path).unwrap();
    assert!(scroll_content.contains("Scroll < CognitiveThought."));
    assert!(scroll_content.contains("biblical_anchor: < \"Genesis 1:3"));

    // 5. Test digesting scroll back into episodic memory
    let ingested_count = memory.digest_scrolls().expect("Digest scrolls");
    assert!(ingested_count >= 1, "Must digest newly inscribed scroll");

    // 6. Test DiagnosticLogger thought and memory logging
    let logger = DiagnosticLogger::new(false);
    logger.log_thought(
        &thought.id,
        &thought.omnicode_tape,
        &thought.premise,
        thought.quad_state.macro_op(),
    );
    logger.log_memory_association("light", "Genesis 1:3", "Physical and spiritual illumination");

    let notes = logger.cognitive_notes();
    assert!(notes.iter().any(|n| n.category == "THOUGHT" && n.tags.contains(&"thought".to_string())));
    assert!(notes.iter().any(|n| n.category == "ASSOCIATION" && n.tags.contains(&"scripture".to_string())));

    // Clean up temporary test directory
    std::fs::remove_dir_all(&temp_dir).ok();
}

/// Verifies that polarity operators (+ Abstract, - Concrete), affixed identifiers
/// (+Dog, -Dog, Dog+, Dog-, +Cell, PROCEED-), and freeform boundary casts ([+], [-], <+>, <->, {+}, {-})
/// lex, parse, and evaluate with zero errors.
#[test]
fn test_polarity_and_boundary_casting_end_to_end() {
    let (logger, debugger) = create_unit_test_fixtures();

    // 1. Lexing Verification
    let source = "+Dog -Dog Dog+ Dog- +Cell PROCEED- [+] [-] <+> <-> {+} {-}";
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize().expect("Tokenization of polarity operators must succeed");

    // Verify +Dog
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::AffixedIdent { name, polarity, position } => {
            name == "Dog" && *polarity == omnicode::core::operators::Polarity::Abstract && *position ==
omnicode::core::operators::AffixPosition::Prefix
        }
        _ => false,
    }));

    // Verify Dog-
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::AffixedIdent { name, polarity, position } => {
            name == "Dog" && *polarity == omnicode::core::operators::Polarity::Concrete && *position ==
omnicode::core::operators::AffixPosition::Postfix
        }
        _ => false,
    }));

```

```
    // Verify PROCEED-
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::AffixedIdent { name, polarity, position } => {
            name == "PROCEED" && *polarity == omnicode::core::operators::Polarity::Concrete && *position ==
omnicode::core::operators::AffixPosition::Postfix
        }
        _ => false,
    }));

    // Verify [+] (Denotation Abstract Cast)
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::BoundaryCast { boundary, polarity } => {
            *boundary == omnicode::core::boundaries::BoundaryType::Denotation && *polarity ==
omnicode::core::operators::Polarity::Abstract
        }
        _ => false,
    }));

    // Verify <-> (Invariant Concrete Cast)
    assert!(tokens.iter().any(|t| match &t.token_type {
        omnicode::parser::lexer::TokenType::BoundaryCast { boundary, polarity } => {
            *boundary == omnicode::core::boundaries::BoundaryType::Invariant && *polarity ==
omnicode::core::operators::Polarity::Concrete
        }
        _ => false,
    }));

    // 2. Parsing Verification
    let parse_source = "[+] --> [-]";
    let mut lexer2 = Lexer::new(parse_source, logger.clone());
    let tokens2 = lexer2.tokenize().unwrap();
    let mut parser2 = Parser::new(tokens2, debugger.clone());
    let prog = parser2.parse_program().expect("Parsing [+] --> [-] must succeed");
    assert_eq!(prog.statements.len(), 1);

    // 3. Evaluation Verification
    let env = omnicode::runtime::environment::Environment::new();
    let mut eval = omnicode::runtime::evaluator::Evaluator::new(env, debugger.clone());

    // Evaluate cast transduction
    let res_cast = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog)).expect("Eval cast transduction");
    assert_eq!(res_cast, omnicode::runtime::environment::RuntimeValue::Text("[-]".to_string()));

    // Evaluate macro keyword with concrete postfix: commit_status [ PROCEED- ]
    let macro_source = "commit_status [ PROCEED- ]";
    let mut lexer3 = Lexer::new(macro_source, logger);
    let tokens3 = lexer3.tokenize().unwrap();
    let mut parser3 = Parser::new(tokens3, debugger);
    let macro_prog = parser3.parse_program().expect("Parsing macro roll");
    let res_macro = eval.eval(&omnicode::ast::nodes::ASTNode::Program(macro_prog)).expect("Eval macro roll");
    assert_eq!(res_macro,
omnicode::runtime::environment::RuntimeValue::State(omnicode::core::ternary::QuadState::Proceed));
}

/// Asserts the physical Operating System primitives of OmniCode:
/// 1. The 13 Silicon Registers in RegisterBank (`HEAD`, `HOME`, `SPOOL`, `IT`, `SELF`, `CALLER`, `OTHER`, `STATE`,
`TURN`, `BOUND`, `FROM`, `INTO`, `DISCARD`).
/// 2. Discard bit-bucket semantics (`DISCARD` drops mutations).
/// 3. Procedural call stack frames on `SPOOL`.
/// 4. Transactional snapshots and rollbacks of physical register states.
/// 5. Directional vector streaming into registers (`10 --> BOUND`, `IT --> INTO`, `FROM --> IT`).
/// 6. Subtractive balanced-ternary parity verification (`!= even`, `== even`).
#[test]
fn test_os_silicon_registers_and_call_frames() {
    let (logger, debugger) = create_unit_test_fixtures();
    let mut env = Environment::new();

    // 1. Pristine Baseline Registers
    assert_eq!(env.registers.read(MachineRegister::HEAD), RuntimeValue::Number(0.0));
    assert_eq!(env.registers.read(MachineRegister::IT), RuntimeValue::State(QuadState::Proceed));
    assert_eq!(env.registers.read(MachineRegister::TURN), RuntimeValue::Number(0.0));
    assert_eq!(env.registers.read(MachineRegister::BOUND), RuntimeValue::Number(0.0));

    // 2. Direct Register Writes & Discard Sink
    env.registers.write(MachineRegister::BOUND, RuntimeValue::Number(100.0));
    assert_eq!(env.registers.read(MachineRegister::BOUND), RuntimeValue::Number(100.0));

    // Discard sink drops mutations
```

```
    env.registers.write(MachineRegister::DISCARD, RuntimeValue::Number(999.0));
    assert_eq!(env.registers.read(MachineRegister::DISCARD), RuntimeValue::State(QuadState::Proceed));

    // 3. Procedural Call Frames on SPOOL
    env.push_call_frame(RuntimeValue::Text("User".into()), RuntimeValue::Text("Process1".into()), 42);
    assert_eq!(env.registers.read(MachineRegister::SPOOL), RuntimeValue::Number(1.0));
    assert_eq!(env.registers.read(MachineRegister::HEAD), RuntimeValue::Number(42.0));
    assert_eq!(env.registers.read(MachineRegister::CALLER), RuntimeValue::Text("User".into()));

    let frame = env.pop_call_frame().expect("Pop call frame");
    assert_eq!(frame.caller, RuntimeValue::Text("User".into()));
    assert_eq!(env.registers.read(MachineRegister::SPOOL), RuntimeValue::Number(0.0));

    // 4. Transactional Rollback of Silicon Registers
    env.begin_transaction();
    env.registers.write(MachineRegister::TURN, RuntimeValue::Number(7.0));
    assert_eq!(env.registers.read(MachineRegister::TURN), RuntimeValue::Number(7.0));
    env.rollback_transaction().expect("Rollback transaction");
    assert_eq!(env.registers.read(MachineRegister::TURN), RuntimeValue::Number(0.0));

    // 5. Evaluator Directional Streaming to Registers
    let mut eval = Evaluator::new(env, debugger.clone());

    // 10 --> BOUND
    let src1 = "10 --> BOUND";
    let mut lex1 = Lexer::new(src1, logger.clone());
    let prog1 = Parser::new(lex1.tokenize().unwrap(), debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog1)).unwrap();
    assert_eq!(eval.env.registers.read(MachineRegister::BOUND), RuntimeValue::Number(10.0));

    // 42 --> IT then IT --> INTO
    let src2 = "42 --> IT";
    let prog2 = Parser::new(Lexer::new(src2, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog2)).unwrap();
    assert_eq!(eval.env.registers.read(MachineRegister::IT), RuntimeValue::Number(42.0));

    let src3 = "IT --> INTO";
    let prog3 = Parser::new(Lexer::new(src3, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog3)).unwrap();
    assert_eq!(eval.env.registers.read(MachineRegister::INTO), RuntimeValue::Number(42.0));
    assert_eq!(eval.env.output_stream.last(), Some(&RuntimeValue::Number(42.0)));

    // 6. Balanced Ternary Parity Checking
    let src4 = "4 = even";
    let prog4 = Parser::new(Lexer::new(src4, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res4 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog4)).unwrap();
    assert_eq!(res4, RuntimeValue::State(QuadState::Proceed));

    let src5 = "5 != even";
    let prog5 = Parser::new(Lexer::new(src5, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res5 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog5)).unwrap();
    assert_eq!(res5, RuntimeValue::State(QuadState::Proceed));

    let src6 = "5 =! even";
    let toks6 = Lexer::new(src6, logger.clone()).tokenize().unwrap();
    let prog6 = Parser::new(toks6, debugger.clone())
        .parse_program()
        .unwrap();
    let res6 = eval
        .eval(&omnicode::ast::nodes::ASTNode::Program(prog6))
        .unwrap();
    assert_eq!(res6, RuntimeValue::State(QuadState::Proceed));

    let src7 = "42 =~ 42";
    let toks7 = Lexer::new(src7, logger.clone()).tokenize().unwrap();
    let prog7 = Parser::new(toks7, debugger.clone())
        .parse_program()
        .unwrap();
    let res7 = eval
        .eval(&omnicode::ast::nodes::ASTNode::Program(prog7))
        .unwrap();
    assert_eq!(res7, RuntimeValue::State(QuadState::Proceed));
}
```

```
/// Asserts the bidirectional transpilation bridge between:
/// - Compiled, verbose, declarative 4-Block spatial reels (`.omni`)
/// - Interpreted line-by-line, lightweight procedural process ribbons (`.tape`)
#[test]
fn test_bidirectional_transpilation_omni_and_tape() {
    let (logger, debugger) = create_unit_test_fixtures();
    let mut assembler = OmniAssembler::new(logger.clone(), debugger.clone());

    let omni_source = r#"
Module < TranspileTest > ( Transpiler Bridge Verification ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}

Opening {
    requires: [ TruthSpace.T4 ],
    preconditions: [ +1 ]
}

Body [
    roll_1 [ 10 --> BOUND ]
    roll_2 [ 0 --> TURN ]
    roll_3 [ commit_status [ PROCEED ] ]
]

Closing {
    verify: [ roll_1 = yes ],
    commit_status [ PROCEED ]
}
"#;

    // Transpile .omni -> .tape
    let tape_text = assembler.transpile_omni_source_to_tape(omni_source).expect("Transpile omni to tape");
    assert!(tape_text.contains("[Tape.TranspileTest]"));
    assert!(tape_text.contains("roll_1 [ 10 --> BOUND ]"));
    assert!(tape_text.contains("roll_2 [ 0 --> TURN ]"));
    assert!(tape_text.contains("seal_roll [ commit_status [ PROCEED ] ]"));

    // Assemble the lowered tape and transpile back to .omni
    let assembled = assembler.assemble_omni(omni_source).expect("Assemble omni");
    let back_to_omni = transpile_tape_to_omni(&assembled);
    assert!(back_to_omni.contains("Module < TranspileTest >"));
    assert!(back_to_omni.contains("Opening {"));
    assert!(back_to_omni.contains("Body ["));
    assert!(back_to_omni.contains("Closing {"));
    assert!(back_to_omni.contains("commit_status [ PROCEED ]"));
}

/// Test 14: Verifies the 6 foundational Base Block Invariant Types in the Lexicon.
/// Asserts that Character, Text, Number, State, Container, and Site all assemble cleanly,
/// lower into physical Word64 machine containers, and commit in affirmative PROCEED.
#[test]
fn test_base_block_primitive_type_invariants_and_baton_pass() {
    let logger = DiagnosticLogger::new(false);
    let debugger = OmniDebugger::new(logger.clone(), 100);
    let mut asm = OmniAssembler::new(logger, debugger);

    let base_block_tapes = [
        "lexicon/01_nouns/ideas/types/character.tape",
        "lexicon/01_nouns/ideas/types/text.tape",
        "lexicon/01_nouns/ideas/types/number.tape",
        "lexicon/01_nouns/ideas/types/state.tape",
        "lexicon/01_nouns/ideas/types/container.tape",
        "lexicon/01_nouns/ideas/types/site.tape",
    ];

    for path_str in &base_block_tapes {
        let path = omnicode::core::filetype::resolve_track_path(path_str);
        assert!(path.exists(), "Base block tape must exist: {}", path_str);
        let assembled = asm.assemble_file(&path).unwrap_or_else(|e| panic!("Failed to assemble Base Block {}: {}",
path_str, e));
        assert_eq!(assembled.commit_status, QuadState::Proceed, "Base Block {} must seal in PROCEED", path_str);
        assert!(!assembled.instructions.is_empty(), "Base Block {} must synthesize instructions", path_str);
    }
}

/// Test 15: Verifies the dual-substrate Character Lifecycle and End-to-End Terminal Echo.
/// Proves that an input character glyph arriving from the terminal port (FROM ::@ 0x03F8)
```

```
/// executes through character.tape and character.omni, undergoes bijective Radix-3 transduction
/// into five balanced trits, verifies single-cell boundary invariants, projects outward to
/// the display target port (INTO ::@ 0x000B8000 / 0x01), and commits with affirmative +1:PROCEED.
#[test]
fn test_character_tape_and_omni_terminal_echo() {
    let (logger, debugger) = create_unit_test_fixtures();
    let mut asm = OmniAssembler::new(logger.clone(), debugger.clone());

    // 1. Verify that character.tape and character.omni exist and assemble to +1:PROCEED
    let tape_path = omnicode::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
    let omni_path = omnicode::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
    assert!(tape_path.exists(), "character.tape must exist in Track 2 lexicon");
    assert!(omni_path.exists(), "character.omni must exist in Track 2 lexicon");

    let assembled_tape = asm.assemble_file(&tape_path).expect("character.tape must assemble cleanly");
    assert_eq!(assembled_tape.commit_status, QuadState::Proceed);
    assert!(!assembled_tape.instructions.is_empty());

    let assembled_omni = asm.assemble_file(&omni_path).expect("character.omni must assemble cleanly");
    assert_eq!(assembled_omni.commit_status, QuadState::Proceed);
    assert!(!assembled_omni.instructions.is_empty());

    // 2. Set up Runtime Environment with simulated Terminal Input Port
    let mut env = Environment::new();

    // Input character 'A' (ASCII 65)
    let input_char = RuntimeValue::from_char('A');

    // Feed terminal input to FROM port
    env.write_from(input_char.clone());

    // Initialize environment symbols for character execution
    env.set("Glyph", input_char.clone());
    env.set("Cell", RuntimeValue::Cell(omnicode::core::cell::Cell::equilibrium()));
    env.set("Target", RuntimeValue::Text("Terminal Stdout".to_string()));
    env.set("Text", RuntimeValue::Text(String::new()));
    env.set("Tripwire", RuntimeValue::Text("ARMED".to_string()));
    env.set("Status", RuntimeValue::State(QuadState::Proceed));

    let mut eval = Evaluator::new(env, debugger.clone());

    // 3. Execute ROLL 1: Glyph Inscription (Outside-in imposition)
    let roll1_src = "{ Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]";
    let prog1 = Parser::new(Lexer::new(roll1_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res1 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog1)).expect("Eval Roll 1");
    assert_eq!(res1, RuntimeValue::State(QuadState::Proceed));
    assert_eq!(eval.env.get("Character`"), Some(&input_char));

    // 4. Execute ROLL 2: Bijective Radix-3 Transduction (<=>)
    let roll2_src = "[ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5 Trits } ]";
    let prog2 = Parser::new(Lexer::new(roll2_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res2 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog2)).expect("Eval Roll 2");
    assert_eq!(res2, RuntimeValue::State(QuadState::Proceed));
    let cell_a = RuntimeValue::Cell(omnicode::core::cell::Cell::from_natural(b'A').unwrap());
    assert_eq!(eval.env.get("Cell`"), Some(&cell_a));

    // 5. Execute ROLL 3: Comparison & Single-Cell Bounds Verification
    let roll3_src = "{ Character` = Glyph } & { Character` <= [ 1 Cell ] } ! GUARD --> [ Status` = +1:PROCEED ]";
    let prog3 = Parser::new(Lexer::new(roll3_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res3 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog3)).expect("Eval Roll 3");
    assert_eq!(res3, RuntimeValue::State(QuadState::Proceed));
    let status_proceed = RuntimeValue::State(QuadState::Proceed);
    assert_eq!(eval.env.get("Status`"), Some(&status_proceed));

    // 6. Execute ROLL 4: Target Projection to Terminal Output Port (INTO)
    let roll4_src = "[ Character` } --> PROJECT [ Target ] = [ Target` }";
    let prog4 = Parser::new(Lexer::new(roll4_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res4 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog4)).expect("Eval Roll 4");
    assert_eq!(res4, RuntimeValue::State(QuadState::Proceed));

    // Verify Terminal Echo in INTO register and output stream
    assert_eq!(eval.env.read_into(), input_char.clone());
    assert_eq!(eval.env.output_stream.last(), Some(&input_char));

```

```
    // 7. Execute ROLL 5: Continuous Text Sequence Ribbon Transition
    let roll5_src = "[ Character` } --> SEQUENCE [ Text ] = [ Text` ]";
    let prog5 = Parser::new(Lexer::new(roll5_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res5 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog5)).expect("Eval Roll 5");
    assert_eq!(res5, RuntimeValue::State(QuadState::Proceed));
    let text_a = RuntimeValue::Text("A".to_string());
    assert_eq!(eval.env.get("Text`"), Some(&text_a));

    // 8. Execute ROLL 6: Closing Commit Status Verification
    eval.env.set("Verify", RuntimeValue::State(QuadState::Proceed));
    let roll6_src = "{ [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED } --> [ +1:PROCEED ]";
    let prog6 = Parser::new(Lexer::new(roll6_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    let res6 = eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog6)).expect("Eval Roll 6");
    assert_eq!(res6, RuntimeValue::State(QuadState::Proceed));

    // 9. Repeat with dynamic Terminal Character 'K' (ASCII 75)
    let glyph_k = RuntimeValue::from_char('K');
    eval.env.write_from(glyph_k.clone());
    eval.env.set("Glyph", glyph_k.clone());
    let prog1_k = Parser::new(Lexer::new(roll1_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog1_k)).expect("Eval Roll 1 for K");
    let prog4_k = Parser::new(Lexer::new(roll4_src, logger.clone()).tokenize().unwrap(),
debugger.clone()).parse_program().unwrap();
    eval.eval(&omnicode::ast::nodes::ASTNode::Program(prog4_k)).expect("Eval Roll 4 for K");
    assert_eq!(eval.env.read_into(), glyph_k);
}

/// Test 16: Verifies the Modular Logger Subsystem:
/// - Flight Recorder & RollFrame telemetry vectors
/// - In-memory circular buffer retention
/// - EpisodicQueryEngine indexing, O(1) lookup & range scanning
/// - Storehouse RotationPolicy enforcement
#[test]
fn test_modular_logger_flight_recorder_query_and_rotation() {
    use omnicode::diagnostics::logger::{
        DiagnosticLogger, RollFrame, RotationPolicy, EpisodicQueryEngine,
    };
    use omnicode::core::ternary::QuadState;

    let logger = DiagnosticLogger::new(false);

    // 1. Inscribe RollFrames into Flight Recorder
    let f1 = RollFrame::new(
        100, 1, 3, "INSCRIBE", (0xFF, 0x00), (0xFF, 0x00),
        QuadState::Proceed, 0, Some("Inscribe glyph".to_string()),
    );
    let f2 = RollFrame::new(
        250, 2, 3, "TRANSDUCE", (0xFF, 0x00), (0x55, 0xAA),
        QuadState::Proceed, 0, Some("Transduce trits".to_string()),
    );
    let f3 = RollFrame::new(
        400, 3, 3, "GUARD", (0x55, 0xAA), (0x00, 0xFF),
        QuadState::Rollback, 1, Some("Guard tripped".to_string()),
    );

    logger.record_roll_frame(f1);
    logger.record_roll_frame(f2);
    logger.record_roll_frame(f3);

    let recent = logger.recent_roll_frames(10);
    assert_eq!(recent.len(), 3);
    assert_eq!(recent[0].verb, "INSCRIBE");
    assert_eq!(recent[2].verb, "GUARD");

    // 2. Anomaly frame detection
    let anomalous = logger.last_anomalous_frame().expect("Must find anomaly");
    assert_eq!(anomalous.quad_state, QuadState::Rollback);
    assert_eq!(anomalous.failpoint_mask, 1);

    // 3. Episodic Query Engine on temp directory
    let temp_dir = std::env::temp_dir().join(format!(
        "omnicode_query_test_{}", std::process::id()
    ));
    let scroll_cat_dir = temp_dir.join("scroll").join("character");
    std::fs::create_dir_all(&scroll_cat_dir).unwrap();
```

```
    let scroll_data = r#"
# BLOCK 1: METADATA BLOCK
Scroll < EpistemicTrace.test_sess_01 > ( Test Prompt ) {
    version: < "1.0.0" >,
    timestamp: < "1792000000000000_0000" >,
    session_id: < "test_sess_01" >,
}
# BLOCK 4: CLOSING BLOCK
Closing {
    commit_status [ <PROCEED> ],
}
"#;
    std::fs::write(
        scroll_cat_dir.join("1792000000000000_0000_test_sess_01.scroll"),
        scroll_data,
    ).unwrap();

    let mut q_engine = EpisodicQueryEngine::new();
    let indexed_count = q_engine.index_directory(&temp_dir).unwrap();
    assert_eq!(indexed_count, 1);

    let session = q_engine
        .find_by_session("test_sess_01")
        .expect("Find session");
    assert_eq!(session.final_state, QuadState::Proceed);
    assert_eq!(session.category, "character");

    // Multi-key O(1) alias lookups
    let s_stem = q_engine
        .find_by_session("1792000000000000_0000_test_sess_01")
        .expect("Find by stem");
    let s_suffix = q_engine
        .find_by_session("0000_test_sess_01")
        .expect("Find by suffix");
    assert_eq!(session.session_id, s_stem.session_id);
    assert_eq!(session.session_id, s_suffix.session_id);

    let proceeds = q_engine.filter_by_state(QuadState::Proceed);
    assert_eq!(proceeds.len(), 1);

    let in_range = q_engine.range_scan(1791000000000000, 1793000000000000);
    assert_eq!(in_range.len(), 1);

    // 4. Rotation Policy enforcement
    let part_dir = temp_dir.join("log").join("number");
    std::fs::create_dir_all(&part_dir).unwrap();

    for i in 0..15 {
        std::fs::write(
            part_dir.join(format!("17920000000000{:02}_item.log", i)),
            "log content",
        ).unwrap();
    }

    let policy = RotationPolicy::new(10, true);
    let rotated = policy.enforce(&part_dir).unwrap();
    assert_eq!(rotated, 5, "Must rotate exactly 5 excess files");

    // Clean up temp dir
    std::fs::remove_dir_all(&temp_dir).ok();
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Suite Verification Proof & Postconditions )
// ============================================================================
// Unit test suite verified and sealed.

```